

Г. В. Табаргина

ОСНОВЫ АЛГОРИТМИЗАЦИИ И ПРОГРАММИРОВАНИЯ НА ЯЗЫКЕ C#

Лабораторный практикум



Министерство науки и высшего образования Российской Федерации
Федеральное государственное бюджетное образовательное учреждение
высшего образования
«Хакасский государственный университет им. Н. Ф. Катанова»

Г. В. Табаргина

ОСНОВЫ АЛГОРИТМИЗАЦИИ И ПРОГРАММИРОВАНИЯ НА ЯЗЫКЕ C#

Учебно-методический комплекс по дисциплине

Лабораторный практикум

Абакан
2022



УДК 004.42/.43:004.438*С#(076.532)
ББК 22.18я723-5
Т120

*Печатается по рекомендации Методического совета
и по решению Редакционно-издательского совета
ФГБОУ ВО «Хакасский государственный университет
им. Н. Ф. Катанова»*

Рецензенты: **О. П. Козумбаева**, кандидат педагогических наук, преподаватель информационных дисциплин Колледжа педагогического образования, истории и права Института педагогического образования ФГБОУ ВО «Хакасский государственный университет им. Н. Ф. Катанова»;
Ю. Л. Борисова, преподаватель высшей квалификационной категории ГБПОУ РХ «Хакасский политехнический колледж»

Табаргина, Галина Викторовна

Т120 Основы алгоритмизации и программирования на языке С#: учебно-методический комплекс по дисциплине: лабораторный практикум/ Г. В. Табаргина. – Абакан: Издательство ФГБОУ ВО «Хакасский государственный университет им. Н. Ф. Катанова», 2022. – 104 с.
ISBN 978-5-7810-2253-3

Лабораторный практикум является частью учебно-методического комплекса по учебной дисциплине ОПЦ.04. «Основы алгоритмизации и программирования» специальности 09.02.06. Сетевое и системное администрирование. В пособии содержится теоретический материал по изучаемым разделам, варианты заданий для лабораторных работ, перечень дополнительных задач, предназначенный для аудиторного и самостоятельного решения.

Издание предназначено для студентов, обучающихся по информационным специальностям программ СПО.

УДК 004.42/.43:004.438*С#(076.532)
ББК 22.18я723-5

ISBN 978-5-7810-2253-3

© ФГБОУ ВО «Хакасский государственный университет им. Н. Ф. Катанова», 2022
© Табаргина Г. В., 2022



ОГЛАВЛЕНИЕ

ВВЕДЕНИЕ	5
Глава 1. АЛГОРИТМ. ПРОГРАММА	6
1.1. Понятие алгоритма	6
1.2. Свойства алгоритмов	7
1.3. Способы записи алгоритмов	7
1.4. Алгоритмические структуры	9
1.5. Программирование	10
1.6. Создание проекта в среде <i>Visual Studio</i>	12
Глава 2. БАЗОВЫЕ ЭЛЕМЕНТЫ ЯЗЫКА C#	15
2.1. Состав языка	15
2.2. Типы данных	16
2.3. Переменные и константы	17
2.4. Организация ввода-вывода данных	17
2.5. Операции	18
Глава 3. ЛИНЕЙНЫЕ ВЫЧИСЛИТЕЛЬНЫЕ ПРОЦЕССЫ	21
3.1. Операторы следования	21
3.2. Арифметические выражения	21
3.3. Примеры решения задач	22
3.4. Лабораторная работа 1. Арифметические выражения в программах	25
3.5. Лабораторная работа 2. Линейные вычислительные процессы	26
3.6. Задачи для самостоятельного решения	28
Глава 4. РАЗВЕТВЛЯЮЩИЕСЯ ВЫЧИСЛИТЕЛЬНЫЕ ПРОЦЕССЫ	30
4.1. Операторы ветвления	30
4.2. Условный оператор <i>if</i>	30
4.3. Оператор выбора <i>switch</i>	31
4.4. Примеры решения задач	32
4.5. Лабораторная работа 3. Условный переход в программах	36
4.6. Лабораторная работа 4. Условный переход и множественный выбор в программах	39
4.7. Задачи для самостоятельного решения	42
Глава 5. ЦИКЛИЧЕСКИЕ ВЫЧИСЛИТЕЛЬНЫЕ ПРОЦЕССЫ	43
5.1. Операторы цикла	43
5.2. Цикл с предусловием <i>while</i>	43
5.3. Цикл с постусловием <i>do..while</i>	44
5.4. Цикл с параметром <i>for</i>	45



5.5. Вложенные циклы	45
5.6. Операторы перехода	46
5.7. Примеры решения задач	46
5.8. Лабораторная работа 5. Циклические конструкции	52
5.9. Лабораторная работа 6. Циклические конструкции	54
5.10. Задачи для самостоятельного решения	56
Глава 6. МАССИВЫ	58
6.1. Одномерные массивы	58
6.2. Стандартные операции с одномерными массивами	59
6.3. Массив как объект	63
6.4. Двумерные массивы	64
6.5. Стандартные операции с двумерными массивами	65
6.6. Лабораторная работа 7. Числовые одномерные массивы	67
6.7. Лабораторная работа 8. Числовые одномерные массивы	69
6.8. Лабораторная работа 9. Числовые двумерные массивы	71
6.9. Лабораторная работа 10. Числовые двумерные массивы	72
6.10. Задачи для самостоятельного решения	76
Глава 7. СИМВОЛЫ И СТРОКИ	79
7.1. Символы <code>char</code>	79
7.2. Строковый <code>map string</code>	80
7.3. Строковый <code>map StringBuilder</code>	84
7.4. Лабораторная работа 11. Символы и строки	85
7.5. Лабораторная работа 12. Символы и строки	87
7.6. Задачи для самостоятельного решения	89
Глава 8. МЕТОДЫ	90
8.1. Основные понятия	90
8.2. Примеры решения задач	91
8.3. Перегрузка методов	95
8.4. Лабораторная работа 13. Методы	95
8.5. Лабораторная работа 14. Методы	96
8.6. Задачи для самостоятельного решения	98
РЕКОМЕНДАТЕЛЬНЫЙ БИБЛИОГРАФИЧЕСКИЙ СПИСОК	100
БИБЛИОГРАФИЧЕСКИЙ СПИСОК	101



ВВЕДЕНИЕ

Учебное пособие составлено в соответствии с федеральным государственным образовательным стандартом среднего профессионального образования по специальности 09.02.06. «Сетевое и системное администрирование».

Учебный материал пособия сгруппирован по разделам в соответствии с содержанием рабочей программы дисциплины ОПЦ.04. «Основы алгоритмизации и программирования» специальности 09.02.06. «Сетевое и системное администрирование».

Учебная дисциплина «Основы алгоритмизации и программирования» является общепрофессиональной дисциплиной, формирующей базовый уровень знаний для освоения профессиональных модулей.

В каждом разделе пособия изложен изучаемый теоретический материал, в том числе содержится необходимый справочный материал по основам языка программирования C#, рассматриваются примеры решения задач от построения блок-схем алгоритмов до разработки программного кода. Лабораторные работы по изучаемому курсу содержат варианты практических заданий в соответствии с изучаемой темой. Кроме того, в каждом разделе представлен дополнительный список задач, которые преподаватель может использовать при организации обучения для совместного решения, а также для организации самостоятельной работы обучающихся.

Пособие может быть использовано при организации учебного процесса по первой части дисциплины общепрофессионального цикла ОПЦ.04. «Основы алгоритмизации и программирования» специальности 09.02.07. «Информационные системы и программирование».



Глава 1. АЛГОРИТМ. ПРОГРАММА

1.1. Понятие алгоритма

Появление алгоритмов связывают с зарождением математики. Более 1000 лет назад (в 825 году) ученый из города Хорезма – Абу Джафар Мухаммед Бен Мусса аль-Хорезми создал книгу по математике, в которой описал способы выполнения арифметических действий над многозначными числами.

Само слово «алгоритм» возникло в Европе после перевода на латынь его книги, в которой имя писалось как «Алгоритми». Научное определение понятия алгоритма было дано в 1930 году. Позже математики вносили свои уточнения в это определение.

Алгоритм – описание последовательности действий (план), строгое исполнение которых приводит к решению поставленной задачи за конечное число шагов.

Алгоритм – подробное описание последовательности арифметических и логических действий, расположенных в строгом логическом порядке и позволяющих решить конкретную задачу.

Алгоритм – метод (способ) решения задачи, записанный по определенным правилам, обеспечивающим однозначность его понимания и механического исполнения при всех значениях исходных данных.

Алгоритм – точное предписание, определяющее вычислительный процесс, ведущий от варьируемых начальных данных к искомому результату.

Алгоритм – формальное описание способа решения задачи путем разбиения ее на последовательность элементарных операций. «Формальное» означает, что описание должно быть абсолютно полным и должно учитывать все возможные ситуации, которые могут возникнуть по ходу решения.

Алгоритмизация – процесс разработки алгоритма для решения задачи.



1.2. Свойства алгоритмов

Однозначность – предлагаемые действия должны быть понятны исполнителю, порядок их исполнения должен быть единственно возможным, любая неопределенность или двусмысленность недопустимы.

Массовость – пригодность алгоритма для решения не только данной задачи, а множества родственных задач, относящихся к общему классу.

Детерминированность – повтор результата при повторе исходных данных.

Корректность – способность алгоритма давать правильные результаты решения задачи при различных исходных данных.

Конечность – решение задачи должно быть получено за конечное число шагов алгоритма.

Эффективность – для успешного решения задачи должны использоваться ограниченные ресурсы конкретного компьютера (объем оперативной памяти, быстродействие ...).

Дискретность – решение задачи разбивается на отдельные элементарные действия (шаги).

Выполняя построение алгоритма решения задачи, необходимо соблюдать следующий порядок действий:

1. Выявить исходные данные, результаты, назначить имена переменным.
2. Выбрать метод (порядок) решения задачи.
3. Разбить метод решения задачи на этапы (с учетом возможностей ЭВМ).
4. Изобразить каждый этап в виде соответствующего блока схемы алгоритма и указать стрелками порядок их выполнения.
5. В полученной схеме при любом варианте вычислений необходимо предусмотреть выдачу результатов или сообщений об их отсутствии, обеспечить возможность после выполнения любой операции так или иначе перейти к блоку «Конец».

1.3. Способы записи алгоритмов

Необходимы такие способы записи алгоритмов, которые легко воспринимаются человеком, но являются достаточно строгими, чтобы их потом можно было перевести на язык компьютера.

1. Описательный (словесно-формульный) – алгоритм записывается на естественном, математическом языке.
2. Графический – в виде блок-схем.

Блок-схема – это последовательность блоков, предписывающих выполнение определенных операций, и связей между этими блоками. Внут-

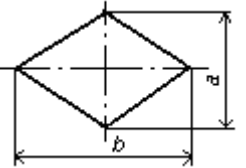
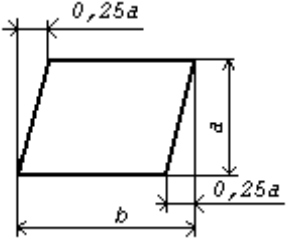


ри блоков указывается информация об операциях, подлежащих выполнению. Конфигурация и размеры блоков, а также порядок графического оформления блок-схем регламентированы ГОСТ 19002-80 и ГОСТ 19003-80 "Схемы алгоритмов и программ".

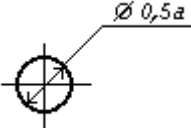
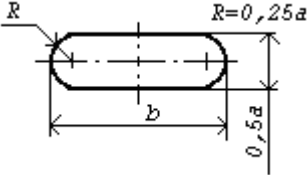
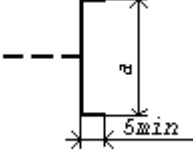
В таблице 1.1 приведены наиболее часто используемые блоки, изображены элементы связей между ними и дано краткое пояснение к ним. Блоки и элементы связей называют элементами блок-схем. При соединении блоков следует использовать только вертикальные и горизонтальные линии потоков. Линии потоков должны быть параллельны линиям внешней рамки или границам листа.

Таблица 1.1

Порядок оформления блок-схем

Наименование	Обозначение и размеры в мм	Функция
1. Процесс		Выполнение операций или группы операций, в результате которых изменяется значение, форма представления или расположение данных
2. Решение		Выбор направления выполнения алгоритма или программы в зависимости от некоторых переменных условий
3. Ввод-вывод		Преобразование данных в форму, пригодную для обработки (ввод) или отображения результатов обработки (вывод)
4. Линия потока		Указание последовательности между символами



5. Соединитель		Указание связи между прерванными линиями потока, связывающими символами
6. Пуск – останов		Начало, конец, прерывание процесса обработки данных или выполнения программы
7. Комментарий		Связь между элементом схемы и пояснением

Расстояние между параллельными линиями потоков должно быть не менее 3 мм, между остальными элементами схемы – не менее 5 мм. Горизонтальный и вертикальный размеры блока должны быть кратны 5 мм (делиться на 5 нацело). Отношение горизонтального и вертикального размеров блока $b/a = 1.5$ является основным. При ручном выполнении блока допустимо отношение $b/a = 2$. Блоки "Начало", "Конец" и "Соединитель" имеют высоту $a/2$, т. е. вдвое меньше основной высоты блоков. Для размещения блоков рекомендуется поле листа разбивать на горизонтальные и вертикальные (для разветвлявшихся схем) зоны.

1.4. Алгоритмические структуры

Следование – линейная последовательность операций, которые выполняются по очереди в порядке записи. Программная реализация такой структуры называется линейной программой. Они предназначены для решения простейших задач, в которых не предусмотрен выбор из нескольких направлений или повторение операций.

Ветвление – предоставляет возможность альтернативного выбора, при выполнении которого алгоритм может пойти по одной из двух возможных ветвей в зависимости от справедливости проверяемого условия.



Обход – представляет собой пропуск нескольких шагов алгоритма при выполнении или невыполнении какого-либо условия.

Цикл – многократно повторяющаяся последовательность шагов алгоритма.

1.5. Программирование

Команды, выполняемые процессором, являются электрическими сигналами, которые можно представить в виде последовательности нулей и единиц. Таким образом, процессор имеет дело с машинным кодом. Написать программу на нем может только опытный программист, знающий архитектуру процессора и его систему команд. Большинство программ создаются при помощи языков программирования высокого уровня.

Программирование – теоретическая и практическая деятельность, связанная с созданием программы.

Язык программирования – совокупность средств и правил представления алгоритма в виде, пригодном для выполнения вычислительной машины.

Язык программирования – формализованный язык для описания алгоритма решения задачи на компьютере.

Программа – это запись (реализация) алгоритма на языке программирования.

Каждый язык программирования имеет алфавит, словарный запас, грамматику, синтаксис и семантику.

Алфавит – фиксированный набор основных символов, допускаемых для составления текста программы.

Синтаксис – правила написания конструкций языка из букв алфавита.

Семантика – правила однозначного толкования отдельных конструкций языка.

Этапы разработки программ

1. Постановка задачи выполняется специалистом в предметной области на естественном языке. Определяется цель задачи, ее содержание и общий подход к решению. Учитываются ограничения, накладываемые аппаратным и программным обеспечением.
2. Анализ задачи и моделирование. Определяются исходные данные и результат, выявляются ограничения на их значения, выполняется формализованное описание задачи и построение (выбор) математической модели, пригодной для ее решения на компьютере.



3. Разработка или выбор алгоритма решения задачи выполняется на основе ее математического описания. Выбирается оптимальное решение. Неточности в постановке, анализе задачи или разработке алгоритма могут привести к скрытой ошибке – программист получит неверный результат, считая его правильным.
4. Проектирование общей структуры программы. Формируется модель решения с последующей детализацией и разбивкой на подпрограммы, определяется «архитектура» программы, способ хранения информации (набор переменных, массивов и т.п.).
5. Кодирование. Запись алгоритма на языке программирования.
6. Отладка и тестирование программы. Отладка – устранение ошибок в программе. Тестирование позволяет вести их поиск и убедиться в том, что отлаженная программа дает правильный результат. Для этого создается система тестов – подобранных примеров с такими наборами данных, для которых решение задачи известно. Тестирование должно охватывать все возможные ветвления в программе, то есть проверять все ее инструкции, и включать такие исходные данные, для которых решение невозможно. Проверка особых ситуаций необходима для анализа корректности. Проверяется устойчивость программы к неумелому обращению пользователя («защита от дурака»). Отладчики – специальные программы, которые позволяют выполнять программу по отдельным шагам, просматривая при этом значения переменных.
7. Анализ результатов. Если программа выполняет моделирование какого-либо известного процесса, следует сопоставить результаты вычислений с результатами наблюдений.
8. Публикация результатов работы, передача заказчику для эксплуатации.
9. Сопровождение программы. Консультации представителей заказчика по работе с программой и обучение персонала. Недостатки и ошибки, замеченные в процессе эксплуатации, должны устраняться.

Типы ошибок в программах

Программирование – процесс творческий, поэтому ошибки неизбежны даже у опытных программистов.

Синтаксические (ошибки компиляции). Возникают при нарушении правил языка, их обнаруживает компилятор, который из-за ошибки не может понять назначение команды.

Ошибки **выполнения** не нарушают синтаксис языка. Они приводят к ошибочным операциям в процессе выполнения программы (попытке де-



ления на ноль, извлечения корня из отрицательного числа). Выполнение программы прерывается.

Семантические (ошибки в алгоритме). При верных исходных данных и внешне безошибочной работе программы приводят к неверным результатам. Они самые коварные и трудные для исправления. Их должен обнаруживать сам программист.

1.6. Создание проекта в среде Visual Studio

Для создания проекта следует запустить VS. Дать команду «Файл – Создать проект», указать тип создаваемого приложения «Консольное приложение», ввести имя проекта и папку для его сохранения. Вид окна приведен на рисунке 1.1.

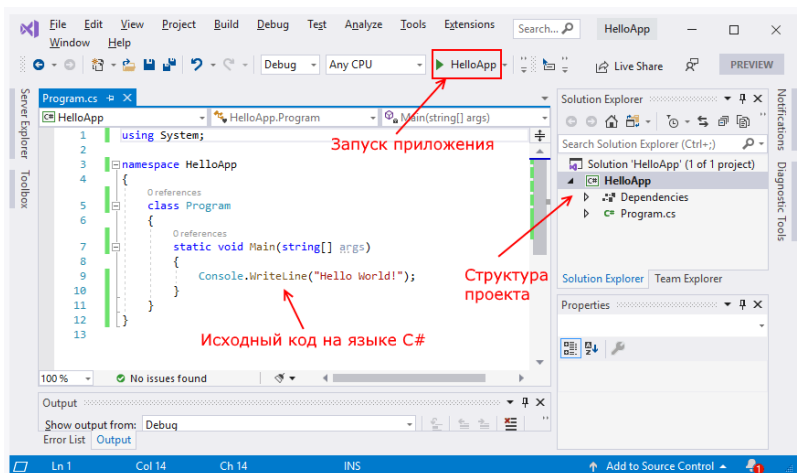


Рис. 1.1. Интерфейс среды VS

В большом поле в центре, которое по сути представляет текстовый редактор, находится сгенерированный по умолчанию код C#. Впоследствии мы изменим его на свой.

Справа находится окно Solution Explorer, в котором можно увидеть структуру нашего проекта. В данном случае у нас сгенерированная по умолчанию структура: узел Properties или Свойств (он хранит файлы свойств приложения и пока нам не нужен); узел Dependencies – это узел содержит сборки dll, которые добавлены в проект по умолчанию. Эти сборки как раз содержат классы библиотеки .NET, которые будет исполь-



зывать C#. Однако не всегда все сборки нужны. Ненужные потом можно удалить, в то же время если понадобится добавить какую-нибудь нужную библиотеку, то именно в этот узел она будет добавляться.

Далее идет непосредственно сам файл кода программы Program.cs. Как раз этот файл и открыт в центральном окне. Вначале разберем, что весь этот код представляет (рис. 1.2):

```
1 using System; // подключаемое пространство имен
2
3 namespace HelloApp // объявление нового пространства имен
4 {
5     class Program // объявление нового класса
6     {
7         static void Main(string[] args) // объявление нового метода
8         {
9             Console.WriteLine("Hello World!"); // действия метода
10        }
11    } // конец объявления нового метода
12
13 } // конец объявления нового класса
14
15 } // конец объявления нового пространства имен
```

Рис. 1.2. Структура кода консольного приложения на C#

В начале файла идет директива `using`, после которой идет название подключаемого пространства имен. Пространства имен представляют собой организацию классов в общие блоки. Например, на первой строке `using System;` подключается пространство имен `System`, которое содержит фундаментальные и базовые классы платформы .NET.

И так как C# имеет Си-подобный синтаксис, каждая строка завершается точкой с запятой, а каждый блок кода помещается в фигурные скобки.

Далее начинается уже собственно наше пространство имен, которое будет создавать отдельную сборку или исполняемую программу: сначала идет ключевое слово `namespace`, после которого название пространства имен. По умолчанию Visual Studio дает ему название проекта. Далее внутри фигурных скобок идет блок пространства имен.

Пространство имен может включать другие пространства или классы. В данном случае у нас по умолчанию сгенерирован один класс – `Program`. Классы объявляются похожим способом – сначала идет ключевое слово `class`, а потом название класса, и далее блок самого класса в фигурных скобках.

Класс может содержать различные переменные, методы, свойства, прочие инструкции. В данном случае у нас объявлен один метод `Main`. В программе на C# метод `Main` является входной точкой программы, с него начинается все управление. Он обязательно должен присутствовать в программе.



Слово `static` указывает, что метод `Main` – статический, а слово `void` – что он не возвращает никакого значения. Позже мы подробнее разберем, что все это значит.

Далее в скобках у нас идут параметры метода – `string[] args` – это массив `args`, который хранит значения типа `string`, то есть строки. В данном случае они нам пока не нужны, но в реальной программе это те параметры, которые передаются при запуске программы из консоли.

Внутри метода располагаются действия, которые этот метод выполняет. По умолчанию он содержит одно действие: `Console.WriteLine("Hello World!");` – оно выводит на консоль строку "Hello World!".

Запустить проект на выполнение можно с помощью клавиши `Ctrl+F5` или нажав на зеленую стрелку.



Глава 2. БАЗОВЫЕ ЭЛЕМЕНТЫ ЯЗЫКА C#

2.1. Состав языка

Алфавит – совокупность допустимых в языке символов. Алфавит языка C# включает:

- прописные и строчные латинские буквы;
- арабские цифры от 0 до 9, шестнадцатеричные цифры от A до F;
- специальные знаки: “ { } | ; [] () + - / % * . \ ‘ : ? < = > ! & ~ ^ @ _ ;
- пробельные символы: пробел, символ табуляции, символ перехода на новую строку.

Из символов алфавита формируются **лексемы** языка: идентификаторы, ключевые (зарезервированные) слова, знаки операций, константы, разделители. Границы лексем определяются другими лексемами (разделителями, знаками операций). В свою очередь лексемы входят в состав выражений и операторов.

Идентификатор – имя программного элемента: константы, переменной, метки, типа, класса, объекта, метода и т.д. Правила составления идентификаторов следующие:

- идентификаторы могут включать латинские буквы, цифры и знак подчеркивания;
- прописные и строчные буквы различаются, например: myname, myName, MyName – три различных идентификатора;
- первым символом может быть буква или символ подчеркивания (но не цифра);
- пробелы или другие разделители внутри имен не допускаются;
- длина идентификатора не ограничивается.

Программным элементам следует давать осмысленные имена, составленные в соответствии с определенными правилами. Существует несколько видов нотаций – соглашений о правилах создания имен:

- нотация Pascal – каждое слово в идентификаторе начинается с заглавной буквы: Age, LastName;
- венгерская нотация – наличие префикса по типу величины: fAge, sName;
- нотация Camel – с заглавной буквы начинается каждое слово в идентификаторе, кроме первого: age, lastName.

Ключевые слова – это зарезервированные идентификаторы, которые имеют специальное значение для компилятора: static, int и т.д. Их можно использовать только по прямому назначению. С полным перечнем ключевых слов и их назначением можно ознакомиться в справочной системе C#.



2.2. Типы данных

C# является языком со строгой типизацией. В нем необходимо объявлять тип всех создаваемых программных элементов (переменных, объектов, окон, кнопок и т.д.). Тип программного элемента сообщает компилятору о его размере и возможностях. В C# типы делятся на три группы:

- базовые типы – предлагаемые языком;
- типы, определяемые пользователем;
- анонимные типы – создаются автоматически на основе инициализации объектов.

Кроме того, типы разбиваются на две другие категории: типы *значения* и *ссылочные* типы. Почти все базовые типы являются типами значениями, исключение составляют типы `object` и `string`, которые являются базовыми, но ссылочными типами данных. Все пользовательские типы, кроме структур, являются ссылочными. Принципиальное различие между типами значениями и ссылочными типами состоит в способе их хранения в памяти. В первом случае фактическое значение хранится в стеке, адрес переменной ссылочного типа тоже хранится в стеке, но сам объект хранится в куче.

Стек – это область памяти, обслуживаемая процессором, в которой хранятся значения локальных переменных. *Куча* – область памяти, используемая для хранения данных, работа с которыми реализуется через указатели и ссылки. Память при этом выделяется программистом динамически, а освобождается сборщиком мусора.

Язык C# предлагает обычный набор базовых типов (табл. 2.1).

Таблица 2.1

Базовые типы данных C#

Тип	Размер в байтах	Описание
1	2	3
object		Может хранить все что угодно, так как является всеобщим предком
<i>Логический тип</i>		
bool	1	true или false
<i>Целочисленные типы</i>		
sbyte	1	Целое со знаком (от -128 до 127)
byte	1	Целое без знака (от 0 до 255)
short	2	Целое со знаком (от -32768 до 32767)
ushort	2	Целое без знака (от 0 до 65535)
int	4	Целое со знаком (от -2147483648 до 2147483647)
uint	4	Целое без знака (от 0 до 4294967295)



1	2	3
long	8	Целое со знаком (от -9223372036854775808 до 9223372036854775807)
ulong	8	Целое без знака (от 0 до 0xffffffff)
<i>Вещественные типы</i>		
float	4	Число с плавающей точкой одинарной точности. Значения приблизительно от $\pm 1,5 \cdot 10^{-45}$ до $\pm 3,4 \cdot 10^{38}$ с 7 значащими цифрами
double	8	Число с плавающей точкой двойной точности. Значения приблизительно от $\pm 5,0 \cdot 10^{-324}$ до $\pm 1,7 \cdot 10^{308}$ с 15-16 значащими цифрами
<i>Символьный тип</i>		
char	2	Символ Unicode
<i>Строковый тип</i>		
string		Строка из Unicode-символов

2.3. Переменные и константы

Переменная представляет собой типизированную область памяти. Разработчик создает переменную, объявляя ее тип и указывая имя. При объявлении переменной ее можно инициализировать (присвоить ей начальное значение), а затем в любой момент ей можно присвоить новое значение, которое заменит собой предыдущее.

Инициализировать каждую переменную сразу при объявлении не обязательно, но необходимо присвоить ей значение до того, как она будет использована.

Константа, в отличие от переменной, не может менять свое значение.

2.4. Организация ввода-вывода данных

Программа при вводе данных и выводе результатов взаимодействует с внешними устройствами. Совокупность стандартных устройств ввода (клавиатура) и вывода (экран) называется консолью. В языке C# для обмена данными с внешними устройствами используются специальные классы. В частности, для работы с консолью используется стандартный класс *Console*, определенный в пространстве имен *System*.

Вывод данных

Вывод данных на экран осуществляет метод *WriteLine*, реализованный в классе *Console*. Существует несколько способов применения данного метода:



```
Console.WriteLine(x);
Console.WriteLine("x=" + x + "y=" + y);
Console.WriteLine("x={0} y={1}",x,y);
Console.WriteLine($"x={x} y={y}");
```

Управление размером поля вывода

Аргументом *WriteLine* указывается строка вида {n, m} – где n определяет идентификатор, а m – количество позиций консоли, отводимых под значение этого идентификатора. При этом значение выравнивается по правому краю поля вывода:

```
Console.WriteLine("{0,20}",x);
Console.WriteLine($" {x,20}");
```

Управление размещением вещественных данных

Аргументом *WriteLine* указывается строка вида {n:##.###}, где n определяет идентификатор, а ##.### определяет формат вещественного числа. В данном случае под целую часть отводится 2 позиции, под дробную – три позиции:

```
Console.WriteLine("{0:##.###}",x);
Console.WriteLine($" {x:##.###}");
```

Ввод данных

Для ввода данных используется метод *ReadLine()*, реализованный в классе *Console*. Данный метод возвращает строковое значение, тип *string*. Чтобы получить числовое значение, необходимо воспользоваться преобразованием данных, используя метод *Parse()* в таких вариантах:

```
double x = double.Parse(Console.ReadLine());
int x = int.Parse(Console.ReadLine());
```

В случае, если соответствующее преобразование выполнить невозможно, то выполнение программы прерывается и генерируется исключение.

2.5. Операции

Операции можно классифицировать по количеству операндов: унарные – воздействуют на один операнд, бинарные – воздействуют на два операнда, тернарные – воздействуют на три операнда.

Инкремент (++) и декремент (--)

Эти операции имеют две формы записи: префиксную – записывается перед операндом и постфиксную – записывается после операнда. Префиксная операция инкремента (декремента) увеличивает (уменьшает) свой операнд на единицу и возвращает измененное значение как результат. Постфиксные версии этих операций возвращают первоначальное значение операнда, а затем изменяют его.



Операция new

Используется для создания нового объекта. С ее помощью можно создавать как объекты ссылочного типа, так и типов значений.

Отрицание

Арифметическое отрицание (-) меняет знак числа на противоположный. Логическое отрицание (!) – определяет операцию инверсии для логического типа.

Явное преобразование типа

Используется для явного преобразования из одного типа в другой. Формат операции имеет следующий вид:

(<тип>) <выражение>;

Умножение (*), деление (/), деление с остатком (%)

Операции применимы для целочисленных и вещественных типов данных. Для других типов эти операции применимы, если для них возможно неявное преобразование к целым или вещественным типам. Если оба операнда при делении целочисленные, то и результат целочисленный.

Сложение (+) и вычитание (-)

Операции применимы для целочисленных и вещественных типов данных. Для других типов эти операции применимы, если для них возможно неявное преобразование к целым или вещественным типам.

Операции отношения (>, <, <=, >=, ==, !=)

Сравнивают значения левого и правого операндов. Результат операции логического типа: true – если значения совпадают, false – в противном случае.

Логические операции И (&&) ИЛИ (||)

Применяются к операндам логического типа.

Результат логической операции И (&&, конъюнкция, логическое умножение) имеет значение true только тогда, когда оба операнда принимают значение true.

Результат логической операции ИЛИ (||, дизъюнкция, логическое сложение) имеет значение true тогда, когда хотя бы один из операндов принимает значение true.

Условная операция (тернарная)

Формат условной операции: (операнд1) ? операнд2 : операнд3;

Операнд1 – это логическое выражение (его результатом может быть true или false). Если true, то результатом всей операции будет значение операнда2, иначе – операнда3. Фактически условная операция является сокращенной формой условного оператора if.

Операции присваивания: =, +=, -= и т.д.

Формат операции простого присваивания (=): операнд2 = операнд1;

В результате выполнения этой операции вычисляется значение операнда1 и результат записывается в операнд2. Можно связать воедино



сразу несколько операторов присваивания: $a = b = c = 100$. Выражение такого вида выполняется справа налево.

В сложных операциях присваивания, например, при сложении с присваиванием ($+=$) к операнду2 прибавляется операнд1 и результат записывается в операнд2, то есть выражение $c+=a$ является более компактной записью выражения $c = c + a$.



Глава 3. ЛИНЕЙНЫЕ ВЫЧИСЛИТЕЛЬНЫЕ ПРОЦЕССЫ

3.1. Операторы следования

Программа на языке C# состоит из последовательности операторов, каждый из которых определяет законченное описание некоторого действия и заканчивается точкой с запятой. Все операторы можно разделить на четыре группы: операторы следования, операторы ветвления, операторы цикла и операторы перехода.

Операторы следования выполняются в естественном порядке: начиная с первого до последнего. Любое выражение, завершающееся точкой с запятой, можно рассматривать как оператор следования.

Составной оператор или блок представляет собой последовательность операторов, заключенных в фигурные скобки `{ }`. Блок обладает собственной областью видимости: объявленные внутри блока имена доступны только внутри данного блока или блоков, вложенных в него. Составные операторы применяются в том случае, когда правила языка предусматривают наличие только одного оператора, а логика программы требует нескольких инструкций. Если заключить несколько операторов в фигурные скобки, то получится блок, который будет рассматриваться компилятором как единый оператор.

3.2. Арифметические выражения

Арифметическое выражение задает порядок действий над элементами и состоит из операндов (констант, переменных, функций), круглых скобок, знаков операций (табл. 3.1). Результатом арифметического выражения является целое или вещественное число.

Язык C# содержит большое количество встроенных математических методов, которые реализованы в классе *Math* пространства имен *System*. Рассмотрим краткое описание некоторых математических методов, подробнее с ними можно ознакомиться в справочной системе VS или технической документации. Особое внимание нужно обратить на типы операндов и результатов, так как каждый метод может иметь несколько пе-



регруженных версий. Стандартный набор математических функций языка C# приведен в таблице 3.1.

Таблица 3.1

Математические функции языка C#

Название	Описание
Math.Abs(<выражение>)	Модуль
Math.Ceiling(<выражение>)	Округление для большего целого
Math.Cos(<выражение>)	Косинус
Math.E	Число e
Math.Exp(<выражение>)	Экспонента
Math.Floor(<выражение>)	Округление до меньшего целого
Math.Log(<выражение>)	Натуральный логарифм
Math.Log10(<выражение>)	Десятичный логарифм
Math.Max(<выражение1>,<выражение2>)	Максимум из двух значений
Math.Min(<выражение1>,<выражение2>)	Минимум из двух значений
Math.PI	Число π
Math.Pow(<выражение1>,<выражение2>)	Возведение в степень
Math.Round(<выражение>)	Простое округление
Math.Sign(<выражение>)	Знак числа
Math.Sin(<выражение>)	Синус
Math.Tan(<выражение>)	Тангенс
Math.Sqrt(<выражение>)	Квадратный корень

3.3. Примеры решения задач

Задача 1. Вычислить периметр и площадь прямоугольного треугольника, если известны длины его катетов.

Указания по решению задачи. Дано: a , b – катеты прямоугольного треугольника. Найти: P – периметр треугольника, S – площадь треугольника. Дополнительные величины: c – гипотенуза треугольника.

$$\text{Решение: } c = \sqrt{a^2 + b^2}; P = a + b + c; S = \frac{ab}{2}$$

Алгоритм решения задачи носит линейный характер. На рисунке 3.1 изображена блок-схема алгоритма.



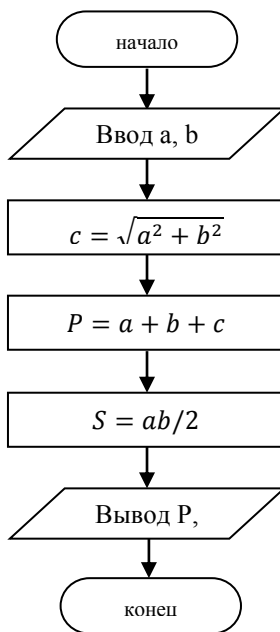


Рис. 3.1. Алгоритм нахождения площади и периметра прямоугольного треугольника

Используемые методы: *WriteLine* – вывод на экран, *ReadLine* – ввод с клавиатуры. Используемый тип данных *double* – вещественный тип (числа с плавающей точкой, дробные). Код программы представлен на рисунке 3.2.

```

static void Main(string[] args)
{
    double a, b, c;
    Console.WriteLine("Введите длины катетов треугольника:");
    Console.Write("a = ");
    a = double.Parse(Console.ReadLine());
    Console.Write("b = ");
    b = double.Parse(Console.ReadLine());
    c = Math.Sqrt(a * a + b * b);
    double P = a + b + c;
    double S = a * b / 2;
    Console.WriteLine("Периметр = " + P);
    Console.WriteLine("Площадь = " + S);
    Console.ReadKey();
}
  
```

Рис. 3.2. Листинг кода

Задача 2. Вычислить сумму цифр трехзначного числа, сгенерированного случайным образом.



Указания по решению задачи. Список величин: x – трехзначное число, сгенерированной случайным образом, x_1 , x_2 , x_3 – отдельные цифры числа x . Выделять цифры из числа будем с помощью операций целочисленного деления $/$ и $\%$. Процесс вычисления носит линейный характер. Листинг кода решения задачи приведен на рисунке 3.3.

```
static void Main(string[] args)
{
    Random rand = new Random();
    int x = rand.Next(100,1000);
    Console.WriteLine("Трехзначное число = "+x);
    int x1, x2, x3;
    x1 = x / 100;
    x3 = x % 10;
    x2 = x % 100 / 10;
    Console.WriteLine("{0} {1} {2}",x1,x2,x3);
    int Summa = x1 + x2 + x3;
    Console.WriteLine("Сумма цифр = "+Summa);
}
```

Рис. 3.3. Листинг кода

Задача 3. Написать программу, проверяющую следующий факт – число, сгенерированное случайным образом, является четным.

Указания по решению задачи. Исходное число x генерируем случайным образом, после чего необходимо вывести его значение в консоль. Используем тот факт, что метод *WriteLine* позволяет выводить значения логических выражений. Для проверки четности нужно найти остаток от деления числа x на два и сравнить его с нулем. Листинг кода решения задачи приведен на рисунке 3.4.

```
static void Main(string[] args)
{
    Random rand = new Random();
    int x = rand.Next(100);
    Console.WriteLine("Случайное число = "+x);
    int ost = x % 2;
    Console.WriteLine("Число четное - {0}",ost==0);
}
```

Рис. 3.4. Листинг кода



3.4. Лабораторная работа 1. Арифметические выражения в программах

Цели: знать математические функции языка программирования C#, правила приоритета арифметических операций, иметь представление об экспоненциальной форме записи вещественных чисел, уметь записывать на языке C# арифметические выражения.

Задание. Разработать программу нахождения значения арифметического выражения для указанных входных данных. Полученный результат сопоставить с заданным в условии.

Значения исходных переменных задать в коде в экспоненциальной форме.

Таблица 3.2

Варианты заданий

Вар-т	Формула	Результат при исходных данных
1	2	3
1	$t = \frac{2 \cos\left(x - \frac{\pi}{6}\right)}{0.5 + \sin^2 y} \left(1 + \frac{z^2}{3 - z^2/5}\right)$	x=14.26 y=-1.22 z=3.5*10 ⁻² t=0.564849
2	$u = \frac{\sqrt[3]{8 + x - y ^2 + 1}}{x^2 + y^2 + 2} - e^{ x-y } (tg^2 z + 1)^x$	x=-4.5 y=0.75*10 ⁻⁴ z=0.845*10 ² U=-55.6848
3	$v = \frac{1 + \sin^2(x + y)}{\left x - \frac{2y}{1 + x^2 y^2}\right } x^{ y } + \cos^2\left(\arctg \frac{1}{z}\right)$	x=3.74*10 ⁻² y=-0.825 z=0.16*10 ² V=1.0553
4	$w = \cos x - \cos y ^{(1+2\sin^2 y)} \left(1 + z + \frac{z^2}{2} + \frac{z^3}{3} + \frac{z^4}{4}\right)$	x=0.4*10 ⁴ y=-0.875 z=-0.475*10 ⁻³ W=1.9873
5	$\alpha = \ln\left(y^{-\sqrt{ x }}\right) \left(x - \frac{y}{2}\right) + \sin^2 \arctg(z)$	x=-15.246 y=4.642*10 ⁻² z=20.001*10 ² α=-182.036
6	$\beta = \sqrt{10(\sqrt[3]{x} + x^{y+2})} (\arcsin^2 z - x - y)$	x=16.55*10 ⁻³ y=-2.75 z=0.15 β=-40.63069
7	$\gamma = 5 \arctg(x) - \frac{1}{4} \arccos(x) \frac{x + 3 x - y + x^2}{ x - y z + x^2}$	x=0.1722 y=6.33 z=3.25*10 ⁻⁴ γ=-205.305



1	2	3
8	$\varphi = \frac{e^{ x-y } x-y ^{x+y}}{\arctg(x) + \arctg(z)} + \sqrt[3]{x^6 + \ln^2 y}$	x=2.235*10 ⁻² y=2.23 z=15.221 φ=39.374
9	$r = \left x^{\frac{y}{x}} - \sqrt[3]{\frac{y}{x}} \right + (y-x) \frac{\cos y - \frac{z}{y-x}}{1 + (y-x)^2}$	x=1.825*10 ² y=18.225 z=-3.298*10 ⁻² R=1.2131
10	$b = y^{\sqrt[3]{ x }} + \cos^3(y) \frac{ x-y \left(1 + \frac{\sin^2 z}{\sqrt{x+y}} \right)}{e^{ x-y } + \frac{x}{2}}$	x=6.251 y=0.827 z=25.001 B=0.7121
11	$h = \frac{x^{y+1} + e^{y-1}}{1 + x y - \operatorname{tg} z } (1 + y-x) + \frac{ y-x ^2}{2} - \frac{ y-x ^3}{3}$	x=2.444 y=0.869*10 ⁻² z=-0.13*10 ³ H=-0.49871
12	$g = \frac{y^{x+1}}{\sqrt[3]{ y-2 } + 3} + \frac{x + \frac{y}{2}}{2 x+y } (x+1)^{-1/\sin z}$	x=12.3*10 ⁻¹ y=15.4 z=0.252*10 ³ G=82.8257
13	$r = \left x^{\frac{y}{x}} - \sqrt[3]{\frac{y}{x}} \right + (y-x) \frac{\cos y - \frac{z}{y-x}}{1 + (y-x)^2}$	x=1.825*10 ² y=18.225 z=-3.298*10 ⁻² R=1.2131
14	$v = \frac{1 + \sin^2(x+y)}{\left x - \frac{2y}{1+x^2y^2} \right } x^{ y } + \cos^2 \left(\arctg \frac{1}{z} \right)$	x=3.74*10 ⁻² y=-0.825 z=0.16*10 ² V=1.0553
15	$\gamma = 5 \arctg(x) - \frac{1}{4} \arccos(x) \frac{x + 3 x-y + x^2}{ x-y z + x^2}$	x=0.1722 y=6.33 z=3.25*10 ⁻⁴ γ=205.305

3.5. Лабораторная работа 2. Линейные вычислительные процессы

Цели: иметь представление о структуре программы на языке C#, реализующей линейные алгоритмы, о числовых типах данных языка C#. Знать и уметь применять инструкции ввода и вывода данных.



Задание. Разработать программы для решения задач в соответствии с вариантом. Значения исходных переменных вводить с клавиатуры (генерировать случайным образом).

Таблица 3.3

Варианты заданий

Вариант	Задания
1	2
1	1. Даны два действительных числа x и y . Вычислить их сумму, разность, произведение и частное 2. Написать программу, которая проверяет следующий факт: сумма цифр данного трехзначного числа N является четным числом.
2	1. Вычислить длину окружности и площадь круга одного и того же заданного радиуса R . 2. Написать программу, которая проверяет следующий факт: квадрат заданного трехзначного числа равен кубу суммы цифр этого числа.
3	1. Найти произведение цифр заданного четырехзначного числа. 2. Написать программу, которая проверяет следующий факт: целое число N является четным двузначным числом.
4	1. Даны два числа. Найти среднее арифметическое кубов этих чисел. 2. Написать программу, которая проверяет следующий факт: треугольник со сторонами a , b , c является равносторонним.
5	1. Дана длина ребра куба. Найти площадь грани, площадь полной поверхности и объем этого куба 2. Написать программу, которая проверяет следующий факт: все цифры данного четырехзначного числа N различны.
6	1. Найти площадь кольца, внутренний радиус которого равен r , а внешний – R ($R > r$). 2. Написать программу, которая проверяет следующий факт: данные числа x , y являются координатами точки, лежащей в первой координатной четверти.
7	1. Написать программу, которая выводит на экран первые четыре степени числа n . 2. Написать программу, которая проверяет следующий факт: число C является средним арифметическим чисел a и b .
8	1. Три сопротивления R_1 , R_2 , R_3 соединены параллельно. Найдите сопротивление соединения. 2. Написать программу, которая проверяет следующий факт: цифры данного трехзначного числа N являются членами арифметической прогрессии.
9	1. Составить программу вычисления объема цилиндра и конуса, которые имеют одинаковую высоту H и одинаковый радиус основания R . 2. Написать программу, которая проверяет следующий факт: в заданном натуральном трехзначном числе N имеется четная цифра.
10	1. Напишите программу вычисления среднего арифметического трех целых чисел, вводимых с клавиатуры (задаваемых случайным образом из промежутка от 0 до 100). 2. Написать программу, которая проверяет следующий факт: сумма каких-либо двух цифр заданного трехзначного натурального числа N равна третьей цифре.



1	2
11	1. Напишите программу вычисления среднего геометрического двух целых чисел, вводимых с клавиатуры (задаваемых случайным образом из промежутка от 0 до 100). Среднее геометрическое двух чисел равно корню квадратному из их произведения. 2. Написать программу, которая проверяет следующий факт: заданное натуральное число N является двузначным и кратным K .
12	1. Написать программу, которая запрашивает расстояние в сантиметрах, а на экран выводит это же расстояние, но в метрах и сантиметрах 2. Написать программу, которая проверяет следующий факт: сумма двух первых цифр заданного четырехзначного числа N равна произведению двух последних.
13	1. Написать программу вычисления площади поверхности цилиндра. Входные данные: радиус основания и высота цилиндра (в сантиметрах). 2. Написать программу, которая проверяет следующий факт: дробь A/B является правильной.
14	1. Написать программу вычисления объема цилиндра. Входные данные: радиус основания и высота цилиндра (в сантиметрах). Работу оформите в цветном окне в центре экрана 2. Написать программу, которая проверяет следующий факт: X – отрицательное целое число, делящееся на K нацело.
15	1. Напишите программу вычисления площади круга, если известна длина окружности. 2. Написать программу, которая проверяет следующий факт: сумма двух последних цифр заданного трехзначного числа N меньше заданного числа K , а первая цифра больше 5.

3.6. Задачи для самостоятельного решения

1. Написать программу вычисления площади поверхности цилиндра. Входные данные: радиус основания и высота цилиндра (в сантиметрах).

2. Написать программу вычисления стоимости покупки, состоящей из нескольких тетрадей и такого же количества обложек к ним. Входные данные: цена тетради, цена обложки (в рублях) и количество комплектов.

3. Заданы координаты трех вершин треугольника (x_1, y_1) , (x_2, y_2) , (x_3, y_3) . Вычислить его периметр и площадь.

4. Напишите программу возведения числа в седьмую степень за четыре операции.

5. Написать программу, которая запрашивает массу тела в граммах, а на экран выводит значение массы в килограммах и граммах.

6. Найдите произведение цифр заданного четырехзначного числа.

7. Написать программу пересчета массы из фунтов в килограммы и граммы. (1 фунт = 409,5 г) Работу программы оформите по образцу:



8. Составить линейную программу, печатающую значение true, если указанное высказывание является истинным, и false – в противном случае. Сумма цифр данного трехзначного числа N является четным числом.

9. Составить линейную программу, печатающую значение true, если указанное высказывание является истинным, и false – в противном случае. Цифры данного четырехзначного числа N образуют строго возрастающую последовательность.

10. Текущее показание электронных часов: t ч ($0 \leq t \leq 23$) n мин ($0 \leq n \leq 59$) k с ($0 \leq k \leq 59$). Какое время будут показывать часы через r ч q мин g с?

11. Написать программу вычисления объема цилиндра. Входные данные: радиус основания и высота цилиндра (в сантиметрах).

12. Написать программу вычисления стоимости покупки, состоящей из нескольких тетрадей и карандашей. Входные данные: цена тетради, цена карандаша (в рублях), количество тетрадей и карандашей.

13. Дана длина ребра куба. Найти площадь грани, площадь полной поверхности и объем куба.

14. Напишите программу возведения числа в четвертую степень за две операции.

15. Написать программу, которая запрашивает расстояние в сантиметрах, а на экран выводит это же расстояние, но в метрах и сантиметрах.

16. Найдите сумму цифр заданного трехзначного числа.

17. Написать программу, которая преобразует введенное с клавиатуры дробное число в денежный формат.

18. Составить линейную программу, печатающую значение true, если указанное высказывание является истинным, и false – в противном случае. Цифры данного трехзначного числа N являются членами арифметической прогрессии.

19. Составить линейную программу, печатающую значение true, если указанное высказывание является истинным, и false – в противном случае. Сумма двух первых цифр заданного четырехзначного числа равна сумме двух его последних цифр.

20. Дано натуральное число T, которое представляет длительность прошедшего времени в секундах. Вывести данное значение длительности в часах, минутах и секундах в следующей форме: HH ч MM мин SS с.



Глава 4. РАЗВЕТВЛЯЮЩИЕСЯ ВЫЧИСЛИТЕЛЬНЫЕ ПРОЦЕССЫ

4.1. Операторы ветвления

Условные операторы и операторы выбора обеспечивают выполнение или невыполнение некоторого оператора (в том числе и составного) в зависимости от справедливости проверяемого условия. Эти операторы прерывают естественную линейную последовательность операций, после чего алгоритм может пойти по одной из двух (или более) возможных ветвей в соответствии с заданным условием. Такая алгоритмическая конструкция называется ветвлением.

Операторы ветвления позволяют изменить порядок выполнения операторов в программе. К операторам ветвления относятся условный оператор *if* и оператор выбора *switch*.

4.2. Условный оператор *if*

Условный оператор *if* используется в ситуации, когда в процессе решения задачи необходимо осуществить проверку какого-либо условия и, в зависимости от его справедливости выполнить либо одно, либо другое действие. Используется для разветвления процесса обработки данных на два направления. Он может иметь одну из форм: *сокращенную* и *полную*.

Сокращенная форма: *if* (*B*) *S*;

Здесь *B* – логическое выражение, истинность которого проверяется, *S* – оператор: простой или составной.

При выполнении сначала вычисляется выражение *B*, затем проводится анализ его результата: если *B* истинно, то выполняется оператор *S*; если *B* ложно, то оператор *S* пропускается. Таким образом, можно либо выполнить оператор *S*, либо пропустить его.

Запись конструкции на языке C# и изображение в блок-схеме приведены на рисунке 4.1.



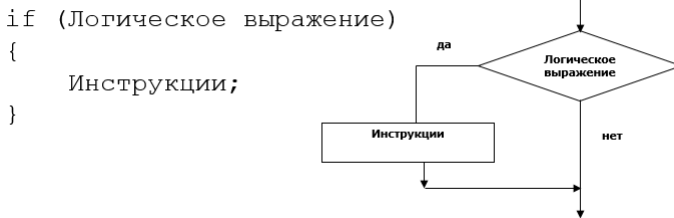


Рис. 4.1. Сокращенная форма оператора *if*

Полная форма: if(B) S1; else S2;

Здесь *B* – логическое выражение, истинность которого проверяется; *S1*, *S2* – оператор: простой или составной.

При выполнении сначала вычисляется значение выражения *B*, затем анализируется его результат: если *B* истинно, то выполняется оператор *S1*, а оператор *S2* пропускается; если *B* ложно, то выполняется оператор *S2*, а *S1* пропускается. Таким образом, с помощью полной формы оператора *if* можно выбрать одно из двух альтернативных действий процесса обработки данных.

Запись конструкции на языке C# и изображение в блок-схеме приведены на рисунке 4.2.

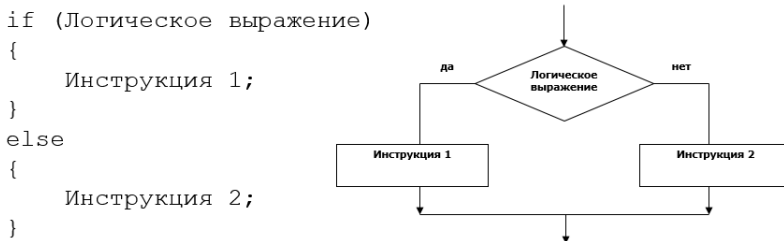


Рис. 4.2. Полная форма оператора *if*

Операторы *S1* и *S2* могут также являться операторами *if*. Такие операторы называются вложенными. При этом ключевое слово *else* связывается с ближайшим предыдущим словом *if*, которое еще не связано ни с одним *else*.

4.3. Оператор выбора *switch*

Предназначен для разветвления процесса вычислений по нескольким направлениям. Формат оператора приведена на рисунке 4.3.




```

switch (<выражение>)
{
    case <константное_выражение_1>:
        [<оператор 1>]; <оператор перехода>;
    case <константное_выражение_2>:
        [<оператор 2>]; <оператор перехода>;
    ...
    case <константное_выражение_n>:
        [<оператор n>]; <оператор перехода>;
    [default: <оператор>;]
}

```

Рис. 4.3. Форма оператора *switch*

Операторы, записанные в квадратных скобках, являются необязательными элементами. Если они отсутствуют, то могут отсутствовать и соответствующие им операторы перехода.

Выражение, за ключевым словом *switch* должно иметь арифметический, символьный, строковый или тип указатель. Все константные выражения должны иметь тип, соответствующий или допускающий неявное преобразование к типу выражения, стоящего после *switch*.

Выполнение оператора начинается с вычисления выражения. Полученный результат сравнивается с меткой *case*. Если результат выражения соответствует метке *case*, то выполняется оператор, стоящий после этой метки, за которым обязательно должен следовать оператор перехода *break*, *goto*. При использовании оператора *break* происходит выход из *switch* и управление передается оператору, следующему за *switch*. Если используется *goto*, то управление передается оператору, помеченному меткой, стоящей после *goto*.

Если необходимо, чтобы для разных меток выполнялось одно и то же действие, то метки перечисляются через двоеточие.

4.4. Примеры решения задач

Задача 1. Написать программу для сравнения двух чисел, введенных с клавиатуры.

Указания по решению задачи. Исходные числа *a* и *b* вводим с клавиатуры. Вариантов развития ситуации может быть три: либо числа равны, либо первое число больше второго, либо первое число меньше второго. Для решения используем два ветвления, одно из которых вложено в другое (рис. 4.4).



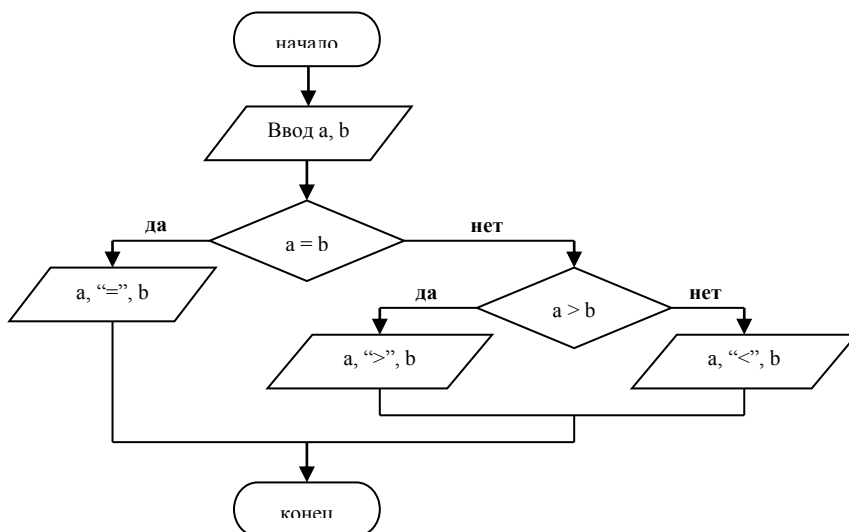


Рис. 4.4. Блок-схема алгоритма сравнения двух чисел

Листинг кода решения задачи сравнения двух чисел приведен на рисунке 4.5. Так как в каждом варианте нужно выполнить только одно действие, инструкцию не обязательно заключать в отдельный программный блок (фигурные скобки {})

```

static void Main(string[] args)
{
    Console.WriteLine("Введите два целых числа:");
    int a, b;
    a = int.Parse(Console.ReadLine());
    b = int.Parse(Console.ReadLine());
    if (a==b)
        Console.WriteLine($"{a} = {b}");
    else if (a>b)
        Console.WriteLine($"{a} > {b}");
    else
        Console.WriteLine($"{a} < {b}");
    Console.ReadKey();
}

```

Рис. 4.5. Листинг программного кода

Задача 2. Написать программу вычисления корней квадратного уравнения $ax^2 + bx + c = 0$, рассмотреть три возможных варианта решения.



Указания по решению задачи. Исходные данные: a , b , c – коэффициенты уравнения. Корни уравнения обозначим x_1 и x_2 , дискриминант – D . Используемые формулы:

$$D = b^2 - 4ac, x_1 = \frac{-b + \sqrt{D}}{2a}, x_2 = \frac{-b - \sqrt{D}}{2a}$$

Вариантов развития ситуации может быть три: два действительных корня, один корень, нет действительных корней. Для решения используем два ветвления, одно из которых вложено в другое (рис. 4.6).

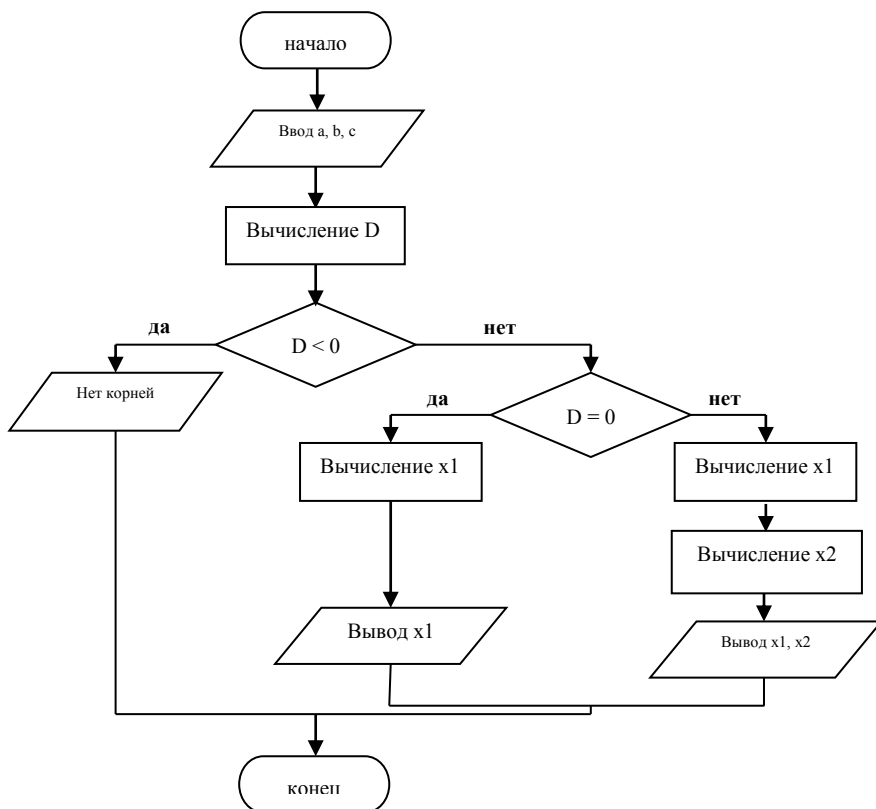


Рис. 4.6. Блок-схема алгоритма решения квадратного уравнения

Листинг программного кода решения квадратного уравнения представлен на рисунке 4.7.



```

static void Main(string[] args)
{
    Console.WriteLine("Введите коэффициенты квадратного уравнения:");
    Console.Write("a = ");
    double a = double.Parse(Console.ReadLine());
    Console.Write("b = ");
    double b = double.Parse(Console.ReadLine());
    Console.Write("c = ");
    double c = double.Parse(Console.ReadLine());
    double x1, x2;
    double D = b * b - 4 * a * c;
    if (D<0)
        Console.WriteLine("Действительных корней нет!");
    else if (D==0)
    {
        x1 = (-b + Math.Sqrt(D)) / (2 * a);
        Console.WriteLine("Уравнение имеет один корень:");
        Console.WriteLine($"x = {x1}");
    }
    else
    {
        x1 = (-b + Math.Sqrt(D)) / (2 * a);
        x2 = (-b - Math.Sqrt(D)) / (2 * a);
        Console.WriteLine("Уравнение имеет два корня:");
        Console.WriteLine($"x1 = {x1}\t x2 = {x2}");
    }
}

```

Рис. 4.7. Листинг кода решения квадратного уравнения

Задача 3. Написать программу, которая генерирует количество яблок от 1 до 20 и выводит на экран это количество со словом «яблоко», «яблока» или «яблок».

Указания по решению задачи. Исходные данные: n – количество яблок. Варианты развития сценария: 1 – яблоко; 2, 3, 4 – яблока; 5-20 – яблок. Генерирование количества яблок реализуем с помощью объекта класса `Random`. Используем его таким образом, чтобы возможное значение находилось в промежутке строго от 1 до 20. Для решения используем оператор множественного выбора `switch`. Варианты для 2, 3 и 4 яблок совпадают по результату, поэтому метки `case` записываются просто через двоеточие. Исполнение программы ограничивается количеством яблок (от 1 до 20). Поэтому ситуация для количества от 5 до 20 обрабатывается блоком `default`. Листинг программного кода представлен на рисунке 4.8.



```

static void Main(string[] args)
{
    Random Rand = new Random();
    int n = Rand.Next(1, 21);
    switch (n)
    {
        case 1:
            Console.WriteLine($"{n} яблоко");
            break;
        case 2:
        case 3:
        case 4:
            Console.WriteLine($"{n} яблока");
            break;
        default:
            Console.WriteLine($"{n} яблок");
            break;
    }
}

```

Рис. 4.8. Листинг кода решения задачи

4.5. Лабораторная работа 3. Условный переход в программах

Цель: знать операции отношения и логические операции языка программирования C#, знать и уметь использовать инструкцию условного перехода языка программирования C#, иметь представление о блоке программного кода.

Задание. Разработать программы для решения задач в соответствии с вариантом. Использовать инструкцию условного перехода if..else.

Таблица 4.1

Варианты заданий

Вариант	Задания
1	- Написать программу, которая проверяет следующий факт: Сумма цифр данного трехзначного числа N является четным числом - Вычислить значение функции $F(x) = \begin{cases} -x^2 + 3x + 9, & \text{если } x \geq 3; \\ \frac{1}{x^3 - 6}, & \text{если } x < 3. \end{cases}$
2	- Написать программу, которая проверяет следующий факт: Сумма двух первых цифр заданного четырехзначного числа равна сумме двух его последних цифр - Вычислить значение функции:



	$F(x) = \begin{cases} x^2 - 3x + 9, & \text{если } x \leq 3; \\ \frac{1}{x^3 + 6}, & \text{если } x > 3. \end{cases}$
3	1. Написать программу, которая проверяет следующий факт: Целое число N является четным двузначным числом 2. Вычислить значение функции $F(x) = \begin{cases} 0, & \text{если } x \leq 1; \\ \frac{1}{x+6}, & \text{если } x > 1. \end{cases}$
4	1. Написать программу, которая проверяет следующий факт: Квадрат заданного трехзначного числа равен кубу суммы цифр этого числа 2. Вычислить значение функции $F(x) = \begin{cases} 9, & \text{если } x \leq -3; \\ \frac{1}{x^2 + 1}, & \text{если } x > -3. \end{cases}$
5	1. Написать программу, которая проверяет следующий факт: Данная тройка натуральных чисел a, b, c является тройкой Пифагора, т.е. c² = a² + b². 2. Вычислить значение функции $F(x) = \begin{cases} x^2 + 4x + 5, & \text{если } x \leq 2; \\ \frac{1}{x^2 + 4x + 5}, & \text{если } x > 2. \end{cases}$
6	1. Написать программу, которая проверяет следующий факт: Треугольник со сторонами a, b, c является равносторонним 2. Найти значение функции $F(x) = \begin{cases} -3x + 9, & \text{если } x \leq 7; \\ \frac{1}{x-7}, & \text{если } x > 7. \end{cases}$
7	1. Написать программу, которая проверяет следующий факт: Все цифры данного четырехзначного числа N различны 2. Найти значение функции $F(x) = \begin{cases} x^2 - x, & \text{если } 0 \leq x \leq 1; \\ x^2 - \sin \pi x^2, & \text{если } x > 1 \text{ или } x < 0. \end{cases}$
8	1. Написать программу, которая проверяет следующий факт: Число c является средним арифметическим чисел a и b 2. Найти значение функции $F(x) = \begin{cases} 4x^2 + 2x - 19, & \text{если } x \geq -3, 5; \\ -\frac{2x}{-4x + 1}, & \text{если } x < -3, 5. \end{cases}$



9	1. Написать программу, которая проверяет следующий факт: Данные числа x, y являются координатами точки, лежащей в первой координатной четверти 2. Найти значение функции $F(x) = \begin{cases} -x^2 + x - 9, & \text{если } x \geq 8; \\ \frac{1}{x^4 - 6}, & \text{если } x < 8. \end{cases}$
10	1. Написать программу, которая проверяет следующий факт: Треугольник со сторонами a, b, c является равнобедренным. 2. Найти значение функции $F(x) = \begin{cases} 3x - 9, & \text{если } x \leq 7; \\ \frac{1}{x^2 - 4}, & \text{если } x > 7. \end{cases}$
11	1. Написать программу, которая проверяет следующий факт: Все цифры данного четырехзначного числа N различны 2. Найти значение функции $F(x) = \begin{cases} x^2 - x, & \text{если } 0 \leq x \leq 1; \\ x^2 - \sin \pi x^2, & \text{если } x > 1 \text{ или } x < 0. \end{cases}$
12	1. Написать программу, которая проверяет следующий факт: В заданном натуральном трехзначном числе N имеется четная цифра. 2. Найти значение функции $F(x) = \begin{cases} 45x^2 + 5, & \text{если } x > 3,6; \\ \frac{5x}{10x^2 + 1}, & \text{если } x \leq 3,6. \end{cases}$
13	1. Написать программу, которая проверяет следующий факт: Цифры данного четырехзначного числа N образуют строго возрастающую последовательность. 2. Найти значение функции $F(x) = \begin{cases} -x^2 + 3x + 9, & \text{если } x \leq 3; \\ \frac{x}{x^2 + 1}, & \text{если } x > 3. \end{cases}$
14	1. Написать программу, которая проверяет следующий факт: Сумма каких-либо двух цифр заданного трехзначного натурального числа N равна третьей цифре. 2. Найти значение функции $F(x) = \begin{cases} x^4 + 9, & \text{если } x < 3,2; \\ \frac{54x^4}{-5x^2 + 7}, & \text{если } x \geq 3,2. \end{cases}$



15	1. Написать программу, которая проверяет следующий факт: Сумма двух последних цифр заданного трехзначного числа N меньше заданного числа K, а первая цифра больше 5. 2. Найти значение функции $F(x) = \begin{cases} -x^2 - 9, & \text{если } x \geq 13; \\ -\frac{1}{x^2 + 9}, & \text{если } x \leq 13. \end{cases}$
----	---

4.6. Лабораторная работа 4. Условный переход и множественный выбор в программах

Цель: знать и уметь использовать инструкцию условного перехода (if..else) и множественного выбора (switch..case) языка программирования C#.

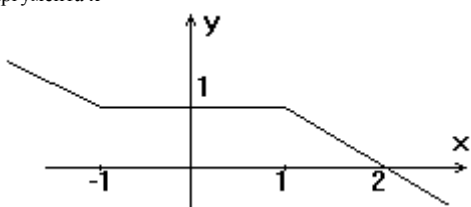
Задание. Разработать программы для решения задач в соответствии с вариантом. Для решения задачи 1 использовать инструкцию if..else, для решения задачи 2 инструкцию switch..case.

Таблица 4.2

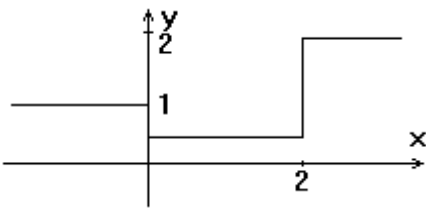
Варианты заданий

Вариант	Задания
1	Три точки на плоскости заданы своими координатами. Выяснить какие из них находятся на максимальном расстоянии друг от друга. Составить программу, которая по номеру семестра печатает курс, к которому относится введенный семестр (1 и 2 семестр – 1 курс, 3 и 4 семестр – 2 курс и т. д.).
2	Вычислить: $\max^3(a, b, c) + \min^2(a, b, c)$ Дано целое число n, соответствующее количеству углов геометрической фигуры. Составить программу, которая по введенному числу n печатает название фигуры (например, при n=3 программа напечатает «треугольник», при n=5 – «пятиугольник», при n>8 – «многоугольник»). В случае если вводится число меньше 2, выводится сообщение об ошибке.
3	Заданы три величины a,b,c. Определить максимальное и минимальное значение из их среднего геометрического, среднего арифметического и суммы экспонент $\exp(a) + \exp(b) + \exp(c)$. Дано целое число в диапазоне от 1 до 5. Вывести строку – словесное описание соответствующей оценки (1 – «плохо», 2 – «неудовлетворительно», 3 – «удовлетворительно», 4 – «хорошо», 5 – «отлично»)
4	Определить в какой четверти координатной плоскости находится точка с координатами x,y.



	Арифметические действия над числами пронумерованы следующим образом: 1 – сложение, 2 – вычитание, 3 – умножение, 4 – деление. Дан номер действия и два числа А и В (В не равно нулю). Выполнить над числами указанное действие и вывести результат
5	$m = \frac{\max(x, y, z)}{\min(x, y)} + 5$ Составьте программу, которая по введенному вами k – числу грибов печатает фразу «Мы нашли в лесу k грибов», причем согласовывает окончание слова «гриб» с k. (Количество грибов может быть любым целым положительным числом: 1, 3, 34, 127 и т. д. Окончание фразы определяется значением последней цифры.)
6	$z = \frac{\min(0, x) - \min(0, y)}{\max^2(y, x)}$ Дано целое число в диапазоне от 0 до 9. Вывести строку – название соответствующей цифры на русском языке (0 – «ноль», 1 – «один», 2 – «два», ...)
7	$r = \frac{\max(\min(x, y), z)}{3}$ Единицы массы пронумерованы следующим образом: 1 – килограмм, 2 – миллиграмм, 3 – грамм, 4 – тонна, 5 – центнер. Дан номер единицы массы и масса тела М в этих единицах (вещественное число). Вывести массу данного тела в килограммах
8	$v = \max(\min(x - y, y - x), 0)$ Единицы длины пронумерованы следующим образом: 1 – дециметр, 2 – километр, 3 – метр, 4 – миллиметр, 5 – сантиметр. Дан номер единицы длины и длина отрезка L в этих единицах (вещественное число). Вывести длину данного отрезка в метрах
9	Вычислить значение функции, заданной графически, по заданному значению аргумента x  Дано целое число в диапазоне 20 – 69, определяющее возраст (в годах). Вывести строку – словесное описание указанного возраста, обеспечив правильное согласование числа со словом «год», например: 20 – «двадцать лет», 32 – «тридцать два года», 41 – «сорок один год».



10	Вычислить значение функции, заданной графически, по заданному значению аргумента x: 
	Элементы окружности пронумерованы следующим образом: 1 – радиус (R), 2 – диаметр (D), 3 – длина (L), 4 – площадь круга (S). Дан номер одного из этих элементов и его значение. Вывести значения остальных элементов данной окружности (в том же порядке). В качестве значения π использовать стандартную константу Pi
11	Решить линейное уравнение $ax + b = 0$ для любых значений коэффициентов.
	Дан номер месяца (1 – январь, 2 – февраль, ...). Вывести название соответствующего времени года («зима», «весна» и т. д.).
12	Проверьте, делится ли выбранное число на 5, на 11 или 13.
	С клавиатуры вводится два целых числа, обозначающих возраст человека и его пол (1 – мужской, 2 – женский). Составить программу, которая в зависимости от введенных данных определяет принадлежность человека к определенной группе: от 0 до 13 – мальчик (девочка); от 14 до 20 – юноша (девушка); от 21 до 70 – мужчина (женщина); более 70 – старец (старушка).
13	Определить попадет ли точка с координатами (x, y) в кольцо с внутренним радиусом $R1$ и внешним радиусом $R2$ с центром в начале координат.
	Элементы равнобедренного прямоугольного треугольника пронумерованы следующим образом: 1 – катет (a), 2 – гипотенуза (c), 3 – высота, опущенная на гипотенузу (h), 4 – площадь (S). Дан номер одного из этих элементов и его значение. Вывести значения остальных элементов данного треугольника (в том же порядке).
14	$w = \max^2(\max(x * y, x + y), 0)$
	Составить программу, вычисляющую площадь геометрической фигуры. Тип фигуры определяется символом (c): О – окружность, Т – равнобедренный прямоугольный треугольник и К – квадрат. Целое число, вводимое вслед за символом, определяет соответствующий элемент для вычисления площади (для окружности это радиус, для треугольника – длина катета, для квадрата – длина стороны).
15	Три точки на плоскости заданы своими координатами. Выяснить какие из них находятся на максимальном расстоянии друг от друга.
	Дано целое число в диапазоне от 1 до 5. Вывести строку – словесное описание соответствующей оценки (1 – «плохо», 2 – «неудовлетворительно», 3 – «удовлетворительно», 4 – «хорошо», 5 – «отлично»).



4.7. Задачи для самостоятельного решения

1. Даны две точки $A(x_1, y_1)$ и $B(x_2, y_2)$. Составить алгоритм, определяющий, которая из точек находится ближе к началу координат.

2. Написать программу, которая выводит пример на умножение двух однозначных чисел, запрашивает ответ пользователя, проверяет его и выводит сообщение «Правильно!» или «Вы ошиблись» и правильный результат. Рекомендуемый вид экрана:

3. Написать программу, которая проверяет, делится ли на пять целое число, введенное с клавиатуры:

4. Написать программу, которая запрашивает длины сторон двух прямоугольников и определяет какой из прямоугольников имеет большую площадь.

5. Написать программу, которая запрашивает возраст человека, а затем, если возраст меньше 14 лет выводит на экран: «У вас пока нет паспорта», если возраст больше 14 лет выводит на экран: «У вас уже есть паспорт», а если возраст равен 14 годам, спрашивает: «Вы получили паспорт?».

6. Написать программу, которая вычисляет стоимость покупки с учетом скидки. Скидка 1% предоставляется, если сумма покупки больше 500 рублей. Информация о предоставленной скидке (процент и величина) должна быть выведена на экран.

7. Написать программу, которая выводит пример на вычитание двух чисел в пределах 100, запрашивает ответ пользователя, проверяет его и выводит сообщение «Правильно!» или «Вы ошиблись» и правильный результат.

8. Написать программу, которая проверяет, делится ли на десять целое число, введенное с клавиатуры.

9. Написать программу, которая запрашивает возраст брата и сестры и сообщает кто старше и на сколько лет или «Они близнецы».

10. Написать программу, которая вычисляет оптимальный вес пользователя, сравнивает его с реальным весом и выдает рекомендации о необходимости поправиться или похудеть. Оптимальный вес вычисляется по формуле: $\text{рост (в см)} - 100$.

11. Написать программу, которая запрашивает номер билета из шести цифр и сообщает «Счастливый» или «Не счастливый» билет попался. Билет считается счастливым, если сумма первых трех цифр равна сумме последних трех цифр.

12. Написать программу, которая запрашивает с клавиатуры три числа (или выбирает их случайным образом) и сообщает, сколько среди них отрицательных.



Глава 5. ЦИКЛИЧЕСКИЕ ВЫЧИСЛИТЕЛЬНЫЕ ПРОЦЕССЫ

5.1. Операторы цикла

Операторы цикла используются для организации многократно повторяющихся вычислений. К операторам цикла относятся: цикл с предусловием *while*, цикл с постусловием *do..while*, цикл с параметром *for* и цикл перебора *foreach*.

подавляющее большинство задач с циклами можно решить разными способами, используя при этом любой из трех первых операторов цикла. Часто решения, использующие разные операторы цикла, оказываются равноценными. В некоторых случаях предпочтительнее использовать какой-то один из операторов. Самым универсальным из всех операторов цикла считается *while*, поэтому в случае затруднений с выбором можно отдать предпочтение ему. Цикл *for* обеспечивает удобную запись циклов с заранее известным числом повторений.

При неумелом использовании циклических конструкций любого типа возможна ситуация, когда компьютер не может нормально закончить цикл (в таком случае говорят, что программа «зациклилась»).

5.2. Цикл с предусловием *while*

Организует выполнение одного оператора (простого или составного) неизвестное заранее число раз. Формат записи: *while (B) S*;

Здесь *B* – логическое выражение, истинность которого проверяется; *S* – тело цикла (простой или составной оператор).

Перед каждым выполнением тела цикла анализируется значение выражения *B*: если оно истинно, то выполняется тело цикла и управление передается на повторную проверку условия *B*; если значение *B* ложно – цикл завершается и управление передается оператору, следующему за оператором *S*.

Если результат выражения *B* окажется ложным при первой проверке, то тело цикла не выполнится ни разу. Если внутри тела цикла не будет инструкции, в результате выполнения которой условие *B* на какой-то итерации станет ложным, то произойдет заикливание, то есть невоз-



возможность выйти из цикла. Поэтому внутри тела должны быть инструкции, приводящие к изменению значения выражения B так, чтобы цикл мог корректно завершиться. Запись конструкции на языке C# и в блок-схеме приведены на рисунке 5.1.

```
while (Логическое выражение)
{
    Тело цикла;
}
```

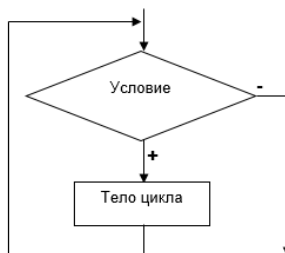


Рис. 5.1. Цикл с предусловием *while*

5.3. Цикл с постусловием *do..while*

Организует выполнение одного оператора (простого или составного) неизвестное заранее число раз. Но в отличие от цикла *while* условие продолжения проверяется после выполнения тела цикла. Формат записи: *do S while (B);*

Здесь B – выражение, истинность которого проверяется; S – тело цикла (простой или составной оператор).

Сначала выполняется оператор S , а затем анализируется значение выражения B : если оно истинно, то управление передается на оператор S , если ложно – цикл завершается и управление передается на оператор, следующий за условием B . Так как условие B проверяется после выполнения тела цикла, то в любом случае тело цикла выполнится хотя бы один раз в любом случае. Запись конструкции на языке C# и в блок-схеме приведены на рисунке 5.2.

```
do
{
    Тело цикла;
}
while (Логическое выражение);
```

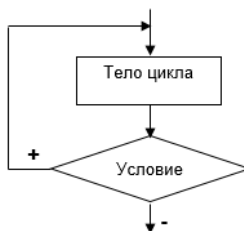


Рис. 5.2. Цикл с постусловием *do while*



При использовании данной конструкции также возможна ситуация заикливания в случае, если условие B всегда будет оставаться истинным.

5.4. Цикл с параметром *for*

Имеет следующую структуру:

`for (<инициализация>; <выражение>; <модификация>) оператор;`

Инициализация используется для объявления и (или) присваивания начального значения величине, используемой в цикле в качестве параметра (счетчика). Областью действия переменной-параметра является цикл и вложенные блоки. Инициализация выполняется один раз в начале исполнения цикла.

Выражение определяет условие выполнения цикла: если его результатом является истина, цикл выполняется. Истинность выражения проверяется перед каждым выполнением тела цикла.

Модификация выполняется после каждой итерации цикла и служит для изменения параметра цикла.

Оператор (простой или составной) представляет собой тело цикла.

Любая из частей оператора `for` может отсутствовать, но точку с запятой, определяющую позицию пропускаемой части, надо оставить. Запись конструкции на языке С# и в блок-схеме приведены на рисунке 5.3.

```
for (int i = 0; i < length; i++)  
{  
    Тело цикла  
}
```



Рис. 5.3. Цикл с параметром *for*

5.5. Вложенные циклы

Циклы могут быть простыми или вложенными. Вложенными могут быть циклы любых типов. Каждый внутренний цикл должен быть полностью вложен во все внешние циклы. Пересечения циклов не допускаются.



5.6. Операторы перехода

Оператор безусловного перехода *goto* передает управление оператору с меткой и имеет формат: *goto* <метка>;

В теле того же метода должна присутствовать ровно одна конструкция вида: <метка>: <оператор>;

Следует учитывать, что использование оператора *goto* затрудняет чтение больших по объему программ, поэтому использовать их нужно только в крайних случаях. Например, внутри инструкции *switch*.

Оператор выхода *break* используется внутри операторов цикла и оператора выбора для обеспечения перехода в точку программы, находящуюся непосредственно за оператором, внутри которого *break* находится. То есть при использовании *break* внутри тела цикла происходит выход из цикла.

Оператор завершения текущей итерации цикла *continue* пропускает все операторы, оставшиеся до конца тела цикла, и передает управление на начало следующей итерации (повторение тела цикла).

5.7. Примеры решения задач

Задача 1. Вывести на экран таблицу квадратов натуральных чисел от 1 до 10.

Указания по решению задачи. Список переменных: *x* – натуральное число, *kv* – квадрат числа. Так как число повторений тела цикла заранее известно (от 1 до 10), то удобно применить в алгоритме решения цикл с параметром. Но реализуем решение задачи тремя способами, с использованием каждого вида цикла.

Способ 1. Цикл *for* (блок-схема алгоритма приведена на рисунке 5.4)

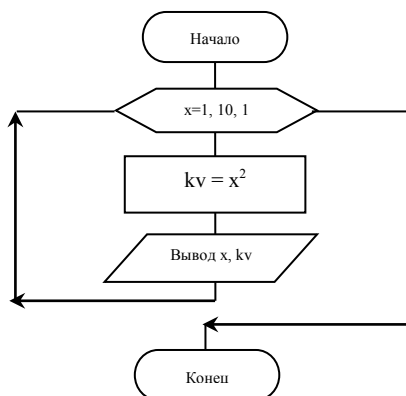


Рис. 5.4. Алгоритм вывода таблицы квадратов (цикл *for*)



Листинг программного кода на языке C# представлен на рисунке 5.5.

```
static void Main(string[] args)
{
    int kv;
    for (int x = 1; x <= 10; x++)
    {
        kv = x * x;
        Console.WriteLine("    {0}    {1}",x,kv);
    }
}
```

Рис. 5.5. Листинг кода вывода таблицы квадратов

Способ 2. Цикл while (блок-схема алгоритма приведена на рисунке 5.6)

В данном способе необходимо в коде управлять переменной x : присвоить начальное значение до цикла, проверить условие продолжения цикла, изменять ее значение на каждой итерации.

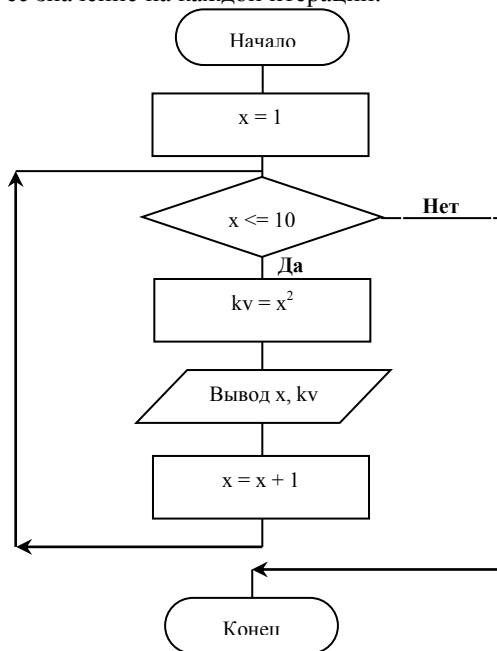


Рис. 5.6. Алгоритм вывода таблицы квадратов (цикл while)

Листинг кода программы вывода на экран таблицы квадратов целых чисел от 1 до 10 на языке C#, реализованной с использованием цикла *while*, представлен на рисунке 5.7.




```

static void Main(string[] args)
{
    int kv, x=1;
    while (x<=10)
    {
        kv = x * x;
        Console.WriteLine("    {0}    {1}", x, kv);
        x++;
    }
}

```

Рис. 5.7. Листинг кода вывода таблицы квадратов

Способ 3. Цикл *do..while* (блок-схема алгоритма приведена на рисунке 5.8)

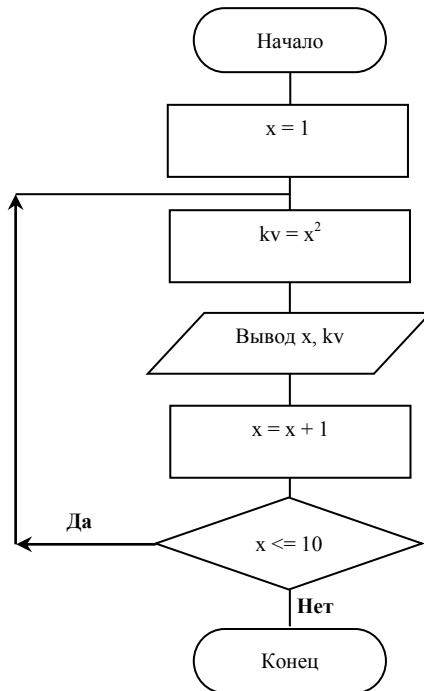


Рис. 5.8. Алгоритм вывода таблицы квадратов (цикл *do..while*)

Листинг кода программы вывода на экран таблицы квадратов целых чисел от 1 до 10 на языке C#, реализованной с использованием цикла *do..while*, представлен на рисунке 5.9.



```

static void Main(string[] args)
{
    int kv, x=1;
    do
    {
        kv = x * x;
        Console.WriteLine("    {0}    {1}",x,kv);
        x++;
    }
    while (x<=10);
}

```

Рис. 5.9. Листинг кода вывода таблицы квадратов

Задача 2. На протяжении N дней выполняется измерение температуры воздуха. Определить: среднее значение температуры, максимальное значение, число дней с температурой ниже нуля.

Листинг программного кода решения задачи на языке C# представлен на рисунке 5.10.

```

static void Main(string[] args)
{
    int N, T, n_day=0, Sum_T=0, max_T=-273;//объявление переменных
    double SR_T;//объявление переменных
    Random rand = new Random();//создание объекта класса Random
    Console.Write("Введите число дней >> ");//вывод пояснения в консоль
    N = int.Parse(Console.ReadLine());//ввод с клавиатуры значения N
    for (int i = 1; i <= N; i++)//заголовок цикла для перебора дней
    {
        T = rand.Next(-40,40);//генерирование текущего значения температуры
        Console.Write("{0} ",T);//вывод в консоль сгенерированного значения T
        Sum_T += T;//добавление значения T в общую сумму
        //поиск максимального значения
        if (T>max_T)
        {
            max_T = T;
        }
        //подсчет числа дней с заморозками
        if (T<0)
        {
            n_day++;
        }
    }
    SR_T = Sum_T / N;//вычисление среднего значения
    Console.WriteLine();
    //вывод в консоль полученных результатов
    Console.WriteLine("Максимальный показатель: {0}",max_T);
    Console.WriteLine("Средний показатель: {0}", SR_T);
    Console.WriteLine("Число дней с заморозками: {0}", n_day);
    Console.ReadKey();
}

```

Рис. 5.10. Листинг программного кода



Указания по решению задачи. Список переменных: N – количество дней, T – значение температуры в текущий день, Sum_T – сумма показаний, SR_T – среднее значение температуры, max_T – максимальное значение температуры, n_day – число дней с отрицательной температурой.

Так как число повторений тела цикла заранее известно (N – вводится с клавиатуры в начале работы программы), то удобно применить в алгоритме решения цикл с параметром.

Задача 3. Вычислить сумму N первых членов ряда

$$S = \frac{1}{1*3} + \frac{1}{2*4} + \frac{1}{3*5} + \dots$$

Указания по решению задачи. Ряд является бесконечным. Количество суммируемых членов ряда вводится с клавиатуры в начале работы программы. Можно выделить два подхода к решению задачи: вывести формулу вычисления произвольного члена ряда по значению его порядкового номера; или обозначить разными переменными величины, входящие в запись члена ряда и управлять ими на каждой итерации.

Список переменных: i – номер члена ряда, R – значение члена ряда, S – вычисляемая сумма ряда, x – вторая величина в знаменателе отдельного члена ряда.

Листинги программного кода решения задачи на языке C# с помощью первого и второго подхода приведены на рисунках 5.11 и 5.12.

```
static void Main(string[] args)
{
    int N = int.Parse(Console.ReadLine());
    double R, S=0;
    for (int i = 1; i <= N; i++)
    {
        R=1.0/(i*(i+2));
        Console.WriteLine("{0}",R);
        S = S + R;
    }
    Console.WriteLine("Сумма {0} членов ряда = {1}",N,S);
}
```

Рис. 5.11. Листинг программного кода (1 способ)



```

static void Main(string[] args)
{
    int N = int.Parse(Console.ReadLine());
    double R, S=0, x=3;
    for (int i = 1; i <= N; i++)
    {
        R=1.0/(i*x);
        Console.WriteLine("{0}",R);
        S += R;
        x++;
    }
    Console.WriteLine("Сумма {0} членов ряда = {1}",N,S);
}

```

Рис. 5.12. Листинг программного кода (2 способ)

Задача 4. Определить, является ли натуральное число, введенное с клавиатуры, простым.

Указания по решению задачи. Натуральное число является простым, если делится только на себя и единицу (имеет два делителя). Соответственно для решения задачи необходимо определить количество делителей данного числа. Для этого будем перебирать диапазон возможных делителей от 1 до самого числа, вычислять остаток от деления числа на возможный делитель, сравнивать остаток с нулем и вести подсчет найденных делителей.

Список переменных: x – вводимое натуральное число, d – возможный делитель числа x, nd – количество делителей числа x. Листинг программного кода решения задачи на языке C# приведен на рисунке 5.13.

```

static void Main(string[] args)
{
    int nd = 0;
    int x = int.Parse(Console.ReadLine());
    for (int d = 1; d <= x; d++)
    {
        if (x % d == 0)
        {
            nd++;
        }
    }
    if (nd==2)
    {
        Console.WriteLine("ПРОСТОЕ!");
    }
    else
    {
        Console.WriteLine("НЕ ЯВЛЯЕТСЯ ПРОСТЫМ!");
    }
}

```

Рис. 5.13. Листинг программного кода



Задача 5. Вывести в консоль таблицу Пифагора (умножения).

Указания по решению задачи. Для выполнения задачи необходимо организовать перебор чисел от 1 до 9. Для каждого из них организовать еще один перебор чисел от 1 до 9. В результате образуется конструкция вложенных циклов. Внутри данной конструкции нужно выполнить умножение первого числа на второе и вывести значение на экран.

Список переменных: i – число по вертикали в первом столбце, j – число по горизонтали в первой строке, R – результат умножения первого числа на второе.

Листинг программного кода решения задачи на языке C# приведен на рисунке 5.14.

```
static void Main(string[] args)
{
    for (int i = 1; i < 10; i++)
    {
        for (int j = 1; j < 10; j++)
        {
            int R = i * j;
            Console.Write("{0}\t ", R);
        }
        Console.WriteLine();
    }
}
```

Рис. 5.14. Листинг программного кода

5.8. Лабораторная работа 5. Циклические конструкции

Цель: знать и уметь использовать инструкции организации циклов языка программирования C#.

Задание 1. Разработать проект в соответствии с вариантом, выполняющий табулирование функции $F(x)$ для x , изменяющегося в интервале от X_1 до X_2 с шагом h . Выполнить поставленную задачу тремя разными способами, используя различные циклические конструкции, сравнить полученные результаты, выполнить проверку результатов вычислений с помощью калькулятора.

Таблица 5.1

Варианты заданий	
Вариант	Задание
1	$F(x) = x - \sin(x)$
2	$F(x) = 2 \cos(x) - 1$
3	$F(x) = \sin^2(x)$



4	$F(x) = \sin(x) - \cos(x)$
5	$F(x) = x \sin(x)$
6	$F(x) = 2 \sin^2(x) + 1$
7	$F(x) = \sin \frac{1}{x} + 2$
8	$F(x) = x \cos\left(\frac{1}{x}\right) + 2$
9	$F(x) = \sqrt{x} \cos^2(x)$
10	$F(x) = 2 \operatorname{tg} \frac{x}{2} + 1$
11	$F(x) = -\cos 2x$
12	$F(x) = \sin x + 0.5 \cos x$
13	$F(x) = 2 \cos \sqrt{x} + 0.5$
14	$F(x) = \frac{x}{\cos x}$
15	$F(x) = \operatorname{tg} 2x - 3$

Задание 2. Вычислить сумму ряда для заданного числа членов. Кроме конечного результата вывести на экран значения каждого члена ряда, участвовавшего в суммировании.

Таблица 5.2

Варианты заданий

Вариант	Задание
1	$s = 1 + \frac{1}{1!} + \frac{1}{2!} + \frac{1}{3!} + \dots$
2	$s = 1 - \frac{1}{1!} + \frac{1}{2!} - \frac{1}{3!} + \dots$
3	$s = 1 + \frac{1}{2} + \frac{1}{4} + \frac{1}{8} + \dots$
4	$s = 1 - \frac{1}{1} + \frac{1}{2} - \frac{1}{3} + \dots$
5	$s = 1 - \frac{1}{2} + \frac{1}{4} - \frac{1}{8} + \dots$
6	$s = \frac{1}{1*3} + \frac{1}{2*4} + \frac{1}{3*5} + \dots$



7	$s = 1 - \frac{1}{1} + \frac{1}{3} - \frac{1}{5} + \dots$
8	$s = \frac{1}{1*2} + \frac{1}{2*3} + \frac{1}{3*4} + \dots$
9	$s = \frac{1}{1*3} + \frac{1}{3*5} + \frac{1}{5*7} + \dots$
10	$s = x + \frac{x^3}{3} + \frac{x^5}{5} + \frac{x^7}{7} + \dots$
11	$s = 1 - x + \frac{x^2}{2!} - \frac{x^3}{3!} + \dots$
12	$s = \frac{1}{2*3} + \frac{2}{3*4} + \frac{3}{4*5} + \dots$
13	$S = \frac{1}{\sin 1} + \frac{1}{\sin 1 + \sin 2} + \dots$
14	$S = 1 \cdot 2 + 2 \cdot 3 \cdot 4 + \dots$

5.9. Лабораторная работа 6. Циклические конструкции

Цель: знать и уметь использовать инструкции организации циклов языка программирования C#.

Задание 1. Найти значение выражения, используя произвольную инструкцию цикла языка программирования (с предусловием, с постусловием, с параметром).

Таблица 5.3

Варианты заданий	
Вариант	Задание
1	$\frac{x}{1} + \frac{(x-1)}{2} + \frac{(x-2)}{4} + \frac{(x-3)}{8} + \frac{(x-4)}{16} + \dots + \frac{1}{2^x}$
2	$\frac{\cos(1)}{\sin(1)} + \frac{\cos(2)}{\sin(1) + \sin(2)} + \dots + \frac{\cos(n)}{\sin(1) + \dots + \sin(n)}$
3	$\sqrt{3 + \sqrt{6 + \dots + \sqrt{3(n-1) + \sqrt{3n}}}}$

4	$\sqrt{3 + \sqrt{6 + \dots + \sqrt{96 + \sqrt{99}}}}$
5	$\frac{\cos(1)}{\sin(n)} * \frac{\cos(2)}{\sin(n-1)} * \dots * \frac{\cos(n)}{\sin(1)}$
6	$1! + 2! + 3! + \dots + n!$
7	$1 + \frac{1}{1!} + \frac{1}{2!} + \frac{1}{3!} + \dots + \frac{1}{n!}$
8	$\left(1 + \frac{1}{1^2}\right) \left(2 + \frac{1}{2^2}\right) \dots \left(n + \frac{1}{n^2}\right)$
9	$\sin(3) + \sin(\sin(3)) + \dots + \sin(\sin(\dots \sin(3)))$
10	$\frac{1}{\sin(1)} + \frac{1}{\sin(1) + \sin(2)} + \dots + \frac{1}{\sin(1) + \dots + \sin(n)}$
11	$\frac{n}{1!} + \frac{n-1}{2!} + \dots + \frac{1}{n!}$
12	$\frac{x}{1} - \frac{(x-1)}{2} + \frac{(x-2)}{4} - \frac{(x-3)}{8} + \frac{(x-4)}{16} - \dots + \frac{1}{2^x}$
13	$\sqrt{n + \sqrt{(n-1) + \dots + \sqrt{(n-2) + \sqrt{1}}}}$
14	$\sqrt{2 + \sqrt{2 + \sqrt{2 + \dots + \sqrt{2}}}}, n \text{ слагаемых}$

Задание 2. Выполнить задание в соответствии с вариантом.

Таблица 5.4

Варианты заданий

Вариант	Задание
1	Найти все трехзначные числа, равные сумме кубов своих цифр.
2	Найти все двузначные числа, сумма цифр которых не меняется при умножении числа на 2,3,4,5,6,7,8,9.
3	Найти все трехзначные числа, сумма цифр которых равна данному целому числу.
4	Найти все трехзначные числа, квадраты которых оканчиваются тремя одинаковыми цифрами, отличными от нуля.
5	В магазине имеется мастика в ящиках по 16 кг, 17 кг и 21 кг. Получить 185 кг мастики без вскрытия ящиков. Рассмотреть все варианты.



6	Представить в записи $42 \cdot 4^*$ вместо звездочек такие цифры, чтобы полученное пятизначное число делилось на 72.
7	Найти все трехзначные числа кратные 7 и сумма цифр которых тоже кратна 7.
8	Найти четырехзначное число, которое при делении на 133 дает в остатке 125, а при делении на 134 дает в остатке 111.
9	Найти сумму делителей данного натурального числа.
10	Сократить дробь, имеющую данные числитель a и знаменатель b . Предусмотреть выделение целой части в случае $a > b$.
11	Определить сколько делителей и какие имеет данное натуральное число.
12	Решить арифметический ребус КИО*ИО= ТОКИО. Вместо каждой буквы необходимо поставить некоторую цифру, причем одинаковые буквы означают одинаковые цифры, а различные буквы – различные цифры.
13	Найти все "пифагоровы" тройки натуральных чисел, наибольшее из которых не превосходит N (Тройка натуральных чисел называется пифагоровой, если сумма квадратов двух из них равна квадрату третьего).
14	Найти все четырехзначные числа, у которых сумма первых двух цифр равна сумме двух последних.

5.10. Задачи для самостоятельного решения

1. Написать программу, которая выводит на экран числа в интервале от 0 до 30 с шагом 3. Числа должны выводиться в столбик. (Задачу решить тремя способами, с применением каждого оператора цикла).

2. Написать программу, которая ищет среди вводимых с клавиатуры чисел максимальное число. Количество чисел предварительно задается с клавиатуры.

3. Написать программу, которая подсчитывает средний вес группы из N человек. Количество человек запрашивается в начале работы программы.

4. Написать программу, которая выводит на экран таблицу умножения на число, введенное с клавиатуры.

5. Написать программу, которая генерирует десять случайных чисел и находит среди них минимальное и максимальное число. Числа могут принимать значения от -40 до $+40$. (Дополнительное задание: на экран выводятся в столбик все десять чисел, минимальное число выводится синим цветом, а максимальное – красным).

6. Вычислить произведение последовательности чисел, вводимых с клавиатуры. Ввод заканчивается нажатием 0. Определить количество введенных чисел.

7. Определить минимальное число из последовательности положительных чисел, вводимых с клавиатуры. Ввод заканчивается нажатием 0.

8. Написать программу, которая ищет среди вводимых с клавиатуры чисел (количество которых предварительно запрашивается с клавиатуры) минимальное число.



9. Написать программу, которая вычисляет факториал натурального числа, введенного с клавиатуры. (Факториалом числа n называется произведение целых чисел от 1 до n . Например, факториал числа 5: $5! = 1 * 2 * 3 * 4 * 5 = 120$).

10. Написать программу, которая генерирует случайные числа, количество которых предварительно запрашивает с клавиатуры и выводит их на экран так, чтобы сначала ставился порядковый номер случайного числа (начиная с 1), а затем само число.

11. Написать программу, которая генерирует десять случайных чисел и находит среди них минимальное и максимальное число. Числа могут принимать значения от -40 до $+40$. (Дополнительное задание: на экран выводятся в столбик все десять чисел, минимальное число выводится синим цветом, а максимальное – красным).

12. Вычислить сумму последовательности чисел, вводимых с клавиатуры. Ввод заканчивается нажатием 0. Определить количество введенных чисел.

13. Определить максимальное число из последовательности положительных чисел, вводимых с клавиатуры. Ввод заканчивается нажатием 0.



Глава 6. МАССИВЫ

Если работа программы связана с хранением и обработкой большого количества однотипных переменных, для их представления в программе можно использовать массивы. Например, пусть программа пользователя выполняет некоторые действия над набором целых чисел, насчитывающей сто элементов, которые требуется сохранять до конца ее работы. Вместо того, чтобы объявлять указанные переменные сто раз, можно один раз объявить целочисленную переменную, состоящую из ста элементов – массив.

Массив представляет собой совокупность данных одного типа с общим для всех элементов именем. Элементы массива пронумерованы, и обратиться к каждому из них можно по номеру. Номера элементов массива иначе называются индексами, а сами элементы массива – переменные с индексами (индексированные переменные). Данные в массивах сохраняются только до конца работы программы. Для их долговременного хранения программа должна записать данные в файл.

К характеристикам массива относятся: тип (общий для всех элементов массива), размерность (количество индексов элемента массива), диапазон изменения индексов (определяет количество элементов в массиве).

6.1. Одномерные массивы

Одномерный массив – это фиксированное количество элементов одного типа, объединенных общим именем, где каждый элемент имеет свой номер. Нумерация элементов массива в C# начинается с нуля.

Одномерный массив в C# реализуется как объект, поэтому его создание представляет собой двухступенчатый процесс. Сначала объявляется ссылочная переменная типа массива, затем выделяется память под требуемое количество элементов базового типа, и ссылочной переменной присваивается адрес нулевого элемента в массиве. Базовый тип определяет тип данных каждого элемента массива. Количество элементов массива определяется его размером.

При необходимости этапы объявления переменной типа массива и выделения памяти могут быть объединены в один. На этапе объявления массива можно произвести его инициализацию. Поэтому для объявления одномерного массива может использоваться одна из следующих форм записи:



```

базовый_тип [] имя_массива;
базовый_тип [] имя_массива = new базовый_тип [размер];
базовый_тип [] имя_массива = {список инициализации};

```

Например:

```

char [] a;
int [] b = new int [10];
double [] c = {0.3, 0.2, -1.2, 31.5};

```

Объявлена ссылка **b** на одномерный массив целых чисел. Выделена память для 10 элементов целого типа, адрес этой области памяти записан в ссылочную переменную **b**. Элементы массива инициализируются по умолчанию нулями.

Объявлена ссылка **c** на одномерный массив вещественных чисел. Выделена память под одномерный массив, размерность которого соответствует количеству элементов в списке инициализации – четырем. Адрес этой области памяти записан в ссылочную переменную **c**. Значение элементов массива соответствует списку инициализации.

Обращение к элементу массива происходит с помощью индекса: **a[0]**, **b[8]**, **c[i]**.

Так как массив представляет собой набор элементов, объединенных общим именем, то обработка массива обычно производится в цикле.

Выход за границы массива в C# расценивается как критическая ошибка, которая ведет к завершению работы программы и генерированию стандартного исключения *IndexOutOfRangeException*.

6.2. Стандартные операции с одномерными массивами

Одномерный массив можно представить, как ряд ячеек, заполненных значениями (рис. 6.1).

Используемые обозначения:

a – одномерный (линейный) массив

n – количество элементов массива

i – индекс (номер) элемента массива

a[i] – элемент массива **a** с индексом **i**

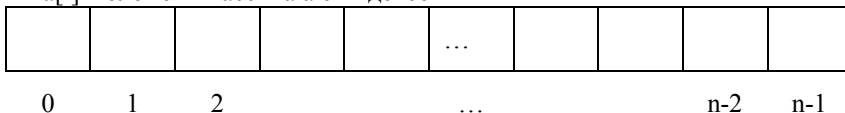


Рис. 6.1. Одномерный массив

Заполнение одномерного числового массива случайными числами (рис. 6.2).



```

Console.Write("Введите число элементов массива:");
int n = int.Parse(Console.ReadLine());
int[] a = new int [n];
Random Rand = new Random();
for (int i = 0; i < n; i++)
{
    a[i] = Rand.Next(-100, 100);
    Console.Write($"{a[i]}\t");
}

```

Рис. 6.2. Заполнение линейного массива случайными числами

Вывод сформированного массива на экран можно реализовать с помощью цикла foreach (рис. 6.3).

```

foreach (int item in a)
{
    Console.Write($"{item}\t");
}

```

Рис. 6.3. Вывод сформированного линейного массива

Вычисление **суммы** всех элементов линейного массива. Добавим следующие переменные в программу: Sum – сумма элементов массива. Последовательно просматривая весь массив, добавим каждый его элемент к общей сумме Sum, в начале работы переменную Sum нужно инициализировать нулевым значением. Блок-схема алгоритма представлена на рисунке 6.4, программный код – на рисунке 6.5.

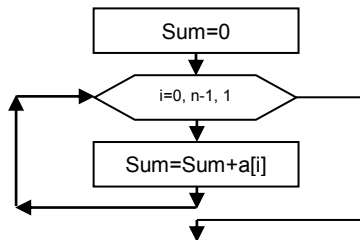


Рис. 6.4. Алгоритм вычисления суммы элементов линейного массива

```

int Sum = 0;
for (int i = 0; i < n; i++)
{
    Sum += a[i];
}
Console.WriteLine($"Сумма элементов = {Sum}");

```

Рис. 6.5. Код вычисления суммы элементов линейного массива

Вычисление **среднего арифметического** всех положительных элементов массива. Добавим следующие переменные в программу: Sp – сумма положительных элементов массива, np- количество положительных элементов, spr – среднее арифметическое положительных



элементов. Просматривая последовательно весь массив, выясняем, является ли элемент положительным, и если да, то добавляем его к общей сумме и счетчик количества положительных элементов увеличиваем на 1. По окончании цикла выполняем деление суммы положительных элементов массива на их количество. Блок-схема алгоритма представлена на рисунке 6.6, программный код – на рисунке 6.7.

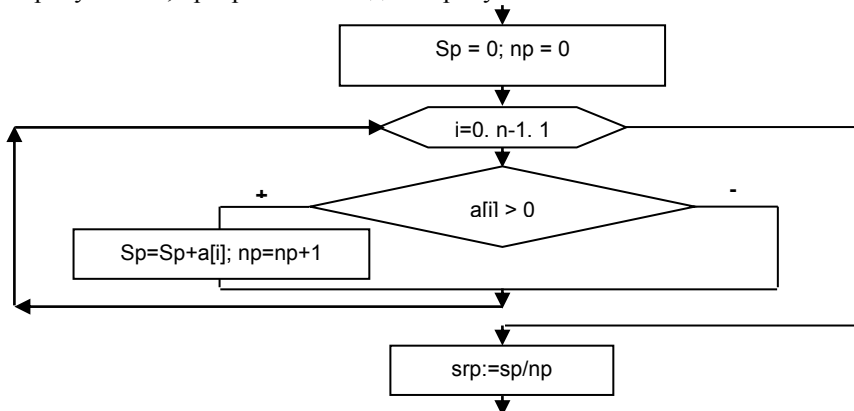


Рис. 6.6. Алгоритм вычисления среднего арифметического положительных элементов линейного массива

```

int Sp = 0, np=0;
double srp;
for (int i = 0; i < n; i++)
{
    if (a[i]>0)
    {
        Sp += a[i]; np++;
    }
}
srp = (double)Sp / np;
Console.WriteLine($"Сумма положительных элементов = {Sp}");
Console.WriteLine($"Ср.арифм-е положительных элементов = {srp}");
  
```

Рис. 6.7. Код вычисления среднего арифметического, положительных элементов массива

Поиск **максимального** элемента линейного массива и его индекса. Добавим в программу следующие переменные: *max* – значение максимального элемента массива, *i_max* – индекс максимального элемента массива. В начале поиска считаем максимальным первый элемент массива (элемент с индексом 0). Последовательно просматривая весь массив, выполняем сравнение каждого элемента с максимальным на текущий момент. Если очередной элемент оказался больше, то присваиваем его значение переменной *max*, а его индекс переменной *imax*. Блок-схема



алгоритма представлена на рисунке 6.8, программный код – на рисунке 6.9.

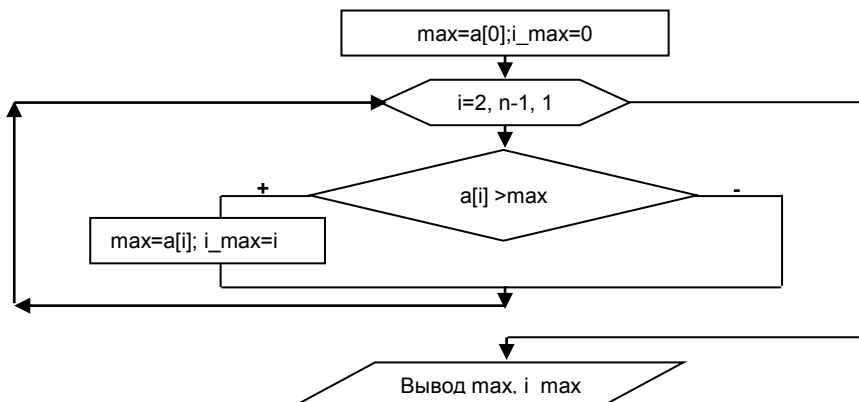


Рис. 6.8. Алгоритм поиска максимального элемента линейного массива

```

int max = a[0];
int i_max = 0;
for (int i = 0; i < n; i++)
{
    if (a[i]>max)
    {
        max = a[i]; i_max = i;
    }
}
Console.WriteLine($"Максимальный элемент a[{i_max}]= {max}");
  
```

Рис. 6.9. Код поиска максимального элемента линейного массива

Перестановка двух элементов линейного массива: первого и максимального. Первый элемент массива имеет индекс 0, иаксимальный элемент в соответствии с предыдущим фрагментом кода имеет номер `i_max`. Для осуществления перестановки необходимо использовать дополнительную переменную в качестве буфера обмена – `buf`. Код фрагмента перестановки элементов приведен на рисунке 6.10.

```

int buf;
buf = a[0];
a[0] = a[i_max];
a[i_max] = buf;
  
```

Рис. 6.10. Код перестановки первого и максимального элементов массива

После осуществления перестановки элементов следует организовать вывод массива на экран для проверки результата.



6.3. Массив как объект

Массивы в C# реализованы как объекты на основе базового класса `Array`, определенного в пространстве имен `System`. Данный класс содержит различные свойства и методы, которые приведены в таблице 6.1. Например, свойство `Length` позволяет определять количество элементов в массиве (рис. 6.11).

```
for (int i = 0; i < a.Length; i++)  
{  
    a[i] = Rand.Next(-100, 100);  
    Console.WriteLine($"{a[i]}\t");  
}
```

Рис. 6.11. Код формирования линейного массива из случайных чисел

Таблица 6.1

Свойства и методы класса `Array`

Элемент	Вид	Описание
<code>BinarySearch</code>	Статический метод	Осуществляет двоичный поиск по ключу в отсортированном массиве
<code>Clear</code>	Статический метод	Присваивает элементам массива значения, определенные по умолчанию (для числовых типов – 0, для ссылочных – null)
<code>Copy</code>	Статический метод	Копирует элементы одного массива в другой массив
<code>CopyTo</code>	экземплярный метод	Копирует элементы текущего одномерного массива в другой массив
<code>IndexOf</code>	Статический метод	Поиск первого вхождения ключевого элемента в одномерный массив. Если элемент найден, то возвращает его индекс, иначе возвращает -1.
<code>LastIndexOf</code>	Статический метод	Поиск последнего вхождения ключевого элемента в одномерный массив. Если элемент найден, то возвращает его индекс, иначе возвращает -1.
<code>Length</code>	Свойство	Возвращает количество элементов в массиве
<code>Rank</code>	Свойство	Возвращает число размерностей массива
<code>Reverse</code>	Статический метод	Изменяет порядок следования элементов в массиве на обратный
<code>Sort</code>	Статический метод	Упорядочивает элементы одномерного массива

Вызов статических методов происходит через обращение к имени класса, например, `Array.Sort(MyArray)`. Обращение к свойству или вызов экземплярного метода производится через обращение к экземпляру класса, например, `MyArray.Length` или `MyArray.GetValue(i)` (рис. 6.12).




```

Array.Sort(a);
Array.Reverse(a);
...
int pos = Array.IndexOf(a, x);
Console.WriteLine($"{x} расположено в массиве в позиции {pos}");

```

Рис. 6.12. Примеры вызова свойств и методов класса `Array`

6.4. Двумерные массивы

Многомерные массивы имеют более одного измерения. Чаще всего используются двумерные массивы, которые представляют собой таблицы.

Каждый элемент массива имеет два индекса, первый определяет номер строки, второй – номер столбца, на пересечении которых находится элемент. Нумерация строк и столбцов начинается с нуля.

Объявить двумерный массив можно одним из предложенных способов:

```

базовый_тип [,] имя_массива;
базовый_тип [,] имя_массива = new базовый_тип [размер1,
размер2];
базовый_тип [,] имя_массива = {{элементы 1-ой
строки}, ..., {элементы n-й строки}};

```

Например:

```

int [,] a;
float [,] b = new float [3,4];
int [,] c = {{0,1,2},{3,4,5}};

```

Объявлена ссылка на двумерный массив целых чисел (имя ссылки *a*), которая в дальнейшем будет использоваться.

Объявлена ссылка *b* на двумерный массив вещественных чисел. Выделена память под 12 элементов, адрес данной области памяти записан в ссылочную переменную *b*. Элементы массива инициализируются по умолчанию нулями.

Объявлена ссылка *c* на двумерный массив целых чисел. Выделена память под 6 элементов. Значения элементов массива соответствуют списку инициализации.

Обращение к элементу массива происходит с помощью индексов: *a[0, 0]*, *b[2, 3]*, *c[i, j]*.

Обработка двумерного массива обычно производится с помощью вложенных циклов. При обращении к свойству *Length* получим общее число элементов в массиве. Чтобы получить число строк, нужно использовать метод *GetLength(0)*, число столбцов – *GetLength(1)*.



6.5. Стандартные операции с двумерными массивами

Двумерный числовой массив представляет собой таблицу, ячейки которой заполнены числовыми значениями (рис. 6.13).

Используемые обозначения:

M – двумерный числовой массив

n_i – количество строк

n_j – количество столбцов

i – номер строки

j – номер столбца

$M[i, j]$ – элемент массива

	0	1	2	...	n_j-1
0					
1					
2					
...					
n_i-1					

Рис. 6.13. Представление двумерного массива

Заполнение числового двумерного массива случайными числами (рис. 6.14). С помощью конструкции вложенных циклов организуем перебор всех элементов массива (внешний цикл перебирает строки, внутренний цикл перебирает столбцы) и присваивание случайного значения элементу. Обращение к отдельному элементу выполняется внутри конструкции вложенных циклов.

```
Console.WriteLine("Введите размер двумерного массива:");
Console.Write("строк ");
int ni = int.Parse(Console.ReadLine());
Console.Write("столбцов ");
int nj = int.Parse(Console.ReadLine());
int[,] M = new int [ni, nj];
Random Rand = new Random();
for (int i = 0; i < ni; i++)
{
    for (int j = 0; j < nj; j++)
    {
        M[i, j] = Rand.Next(-100, 100);
        Console.Write($"{M[i, j]}\t");
    }
    Console.WriteLine();
}
```

Рис. 6.14. Заполнение двумерного массива случайными числами



Вывод двумерного массива на экран с использованием метода *GetLength()* класса *Array* (рис. 6.15).

```
for (int i = 0; i < M.GetLength(0); i++)
{
    for (int j = 0; j < M.GetLength(1); j++)
    {
        Console.Write($"{M[i, j]}\t");
    }
    Console.WriteLine();
}
```

Рис. 6.15. Вывод двумерного массива

Заполнять числовой массив можно не только случайными числами, а в соответствии с некоторой логикой конкретными значениями. Заполним числовой двумерный массив значениями, равными **произведению** номера строки и столбца (рис. 6.16).

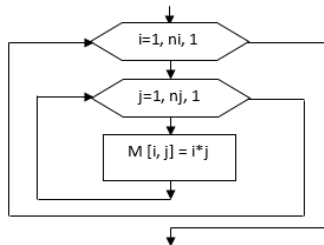


Рис. 6.16. Алгоритм заполнения двумерного массива произведениями номеров строки и столбца

Программная реализация алгоритма представлена на рисунке 6.17.

```
for (int i = 0; i < M.GetLength(0); i++)
{
    for (int j = 0; j < M.GetLength(1); j++)
    {
        M[i, j] = i*j;
        Console.Write($"{M[i, j]}\t");
    }
    Console.WriteLine();
}
```

Рис. 6.17. Код заполнения двумерного массива произведениями номеров строки и столбца

Ряд задач требует выполнить обработку двумерного массива по строкам или по столбцам. Вычислим **суммы** элементов в каждом отдельном столбце двумерного массива. Для решения нужно организовать просмотр элементов двумерного массива по столбцам, перед каждым новым столбцом обнулить сумму, затем в цикле по строкам каждый элемент столбца добавить к общей сумме, по окончании просмотра столбца вывести



накопленную сумму в консоль (рис. 6.18). Добавим в программный код переменную Sum – сумма элементов отдельного столбца.

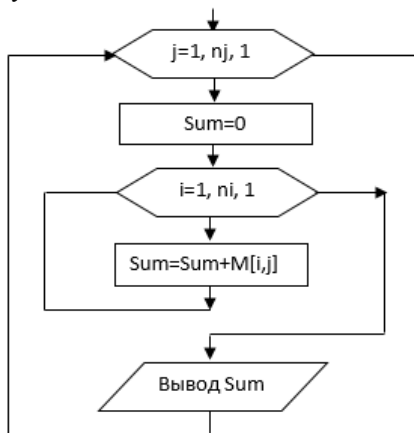


Рис. 6.18. Алгоритм вычисления суммы элементов двумерного массива по столбцам

Листинг программного кода на языке C# вычисления суммы элементов двумерного массива по столбцам представлен на рисунке 6.19.

```

for (int j = 0; j < M.GetLength(1); j++)
{
    int Sum = 0;
    for (int i = 0; i < M.GetLength(0); i++)
    {
        Sum += M[i, j];
    }
    Console.WriteLine($"Сумма {j} столбца = {Sum}");
}
  
```

Рис. 6.19. Код вычисления суммы элементов двумерного массива по столбцам

6.6. Лабораторная работа 7. Числовые одномерные массивы

Цель: иметь представление об организации в памяти одномерных массивов, уметь объявлять числовой одномерный массив, уметь его заполнять, выводить в консоль, обращаться к элементам одномерного массива, уметь выполнять обработку элементов одномерного массива.

Вариант 1

1. Заполнить с клавиатуры одномерный массив из 5 целых чисел и вывести количество ненулевых элементов. Перед вводом каждого элемента на экране должна появляться подсказка с его номером.

A[1] = 12



$A[2] = 0$

$A[3] = 12$

$A[4] = 3$

$A[5] = 0$

В массиве 3 ненулевых элемента.

2. Заполнить массив из 10 элементов случайными числами. Вычислить среднее арифметическое всех его положительных элементов.

Полученный массив:

0 4 -3 2 -7 0 5 -6 3 1

Сумма всех его положительных элементов = 15

Количество положительных элементов = 5

Их среднее арифметическое = 3

3. Заполнить массив из 10 элементов случайными числами. Увеличить каждый его элемент в 2 раза.

Исходный массив: 0 4 -3 2 -7 0 5 -6 3 1

Конечный массив: 0 8 -6 4 -14 0 10 -12 6 2

4. Заполнить линейный массив из 10 элементов случайными числами. Найти минимальный элемент массива и вывести его на экран с пояснениями.

5. Написать программу, которая вычисляет, сколько раз введенное с клавиатуры число встречается в массиве из 10 случайных чисел.

Введите число: 3

Массив имеет вид: 0 4 3 2 7 0 5 6 3 1

Число 3 встречается в массиве 2 раза

6. Заполнить массив случайными числами. Вывести на экран его положительные элементы красным цветом, а отрицательные – синим.

Вариант 2

1. Заполнить с клавиатуры одномерный массив из 5 целых чисел и вывести количество нулевых элементов. Перед вводом каждого элемента на экране должна появляться подсказка с его номером.

$A[1] = 12$

$A[2] = 0$

$A[3] = 12$

$A[4] = 3$

$A[5] = 0$

В массиве 2 нулевых элемента.

2. Заполнить массив из 10 элементов случайными числами. Вычислить среднее арифметическое всех его ненулевых элементов.

Полученный массив:

0 4 -3 2 -7 0 5 -6 3 1

Сумма всех его ненулевых элементов = -1

Количество ненулевых элементов = 8

Их среднее арифметическое = -0.125



3. Заполнить два массива (из 10 элементов каждый) случайными числами. Сформировать третий массив, каждый элемент которого равен полусумме соответствующих элементов исходных массивов.

Исходные массивы:

0 4 -3 2 -7 0 5 -6 3 1

5 2 0 -4 3 8 -3 8 -9 5

Конечный массив:

2.5 3 -1.5 -1 -2 4 1 1 -3 3

4. Заполнить линейный массив из 10 элементов случайными числами. Найти максимальный элемент массива и вывести его на экран с пояснениями.

5. Написать программу, которая вычисляет, сколько раз введенное с клавиатуры число встречается в массиве из 10 случайных чисел.

Введите число: 3

Массив имеет вид: 0 4 3 2 7 0 5 6 3 1

Число 3 встречается в массиве 2 раза

6. Заполнить линейный массив случайными числами. Вывести на экран его положительные элементы красным цветом, а отрицательные – синим.

6.7 Лабораторная работа 8. Числовые одномерные массивы

Цель: иметь представление об организации в памяти одномерных массивов, уметь объявлять числовой одномерный массив, уметь его заполнять, выводить в консоль, обращаться к элементам одномерного массива, уметь выполнять обработку элементов одномерного массива.

Задание. Разработать программу для решения задачи в соответствии с вариантом. Исходный массив заполнить случайными числами.

Таблица 6.2

Варианты заданий

Вариант	Задание
1	В одномерном массиве, состоящем из n целочисленных элементов, вычислить: 1) сумму отрицательных элементов массива; 2) произведение элементов массива, расположенных между максимальным и минимальным элементами. Упорядочить элементы массива по возрастанию.
2	В одномерном массиве, состоящем из n целочисленных элементов, вычислить: 1) сумму положительных элементов массива; 2) произведение элементов массива, расположенных между максимальным по модулю и минимальным по модулю элементами. Упорядочить элементы массива по убыванию.



3	В одномерном массиве, состоящем из n целых элементов, вычислить: 1) произведение элементов массива с четными номерами; 2) сумму элементов массива, расположенных между первым и последним нулевыми элементами. Преобразовать массив таким образом, чтобы сначала располагались все положительные элементы, а потом – все отрицательные (элементы, равные 0, считать положительными).
4	В одномерном массиве, состоящем из n целочисленных элементов, вычислить: 1) сумму элементов массива с нечетными номерами; 2) сумму элементов массива, расположенных между первым и последним отрицательными элементами. Сжать массив, удалив из него все элементы, модуль которых не превышает 1. Освободившиеся в конце массива элементы заполнить нулями.
5	В одномерном массиве, состоящем из n целочисленных элементов, вычислить: 1) максимальный элемент массива; 2) сумму элементов массива, расположенных до последнего положительного элемента. Сжать массив, удалив из него все элементы, модуль которых находится в интервале $[a, b]$. Освободившиеся в конце массива элементы заполнить нулями.
6	В одномерном массиве, состоящем из n целочисленных элементов, вычислить: 1) минимальный элемент массива; 2) сумму элементов массива, расположенных между первым и последним положительными элементами. Преобразовать массив таким образом, чтобы сначала располагались все элементы, равные нулю, а потом – все остальные.
7	В одномерном массиве, состоящем из n целых элементов, вычислить: 1) номер максимального элемента массива; 2) произведение элементов массива, расположенных между первым и вторым нулевыми элементами. Преобразовать массив таким образом, чтобы в первой его половине располагались элементы, стоявшие в нечетных позициях, а во второй половине – элементы, стоявшие в четных позициях.
8	В одномерном массиве, состоящем из n целочисленных элементов, вычислить: 1) количество отрицательных элементов массива; 2) сумму модулей элементов массива, расположенных после минимального по модулю элемента. Заменить все отрицательные элементы массива их квадратами и упорядочить элементы массива по возрастанию.
9	В одномерном массиве, состоящем из n целочисленных элементов, вычислить: 1) максимальный по модулю элемент массива; 2) сумму элементов массива, расположенных между первым и вторым положительными элементами. Преобразовать массив таким образом, чтобы элементы, равные нулю, располагались после всех остальных.



10	В одномерном массиве, состоящем из n целых элементов, вычислить: 1) минимальный по модулю элемент массива; 2) сумму модулей элементов массива, расположенных после первого элемента, равного нулю. Преобразовать массив таким образом, чтобы в первой его половине располагались элементы, стоявшие в четных позициях, а во второй половине – элементы, стоявшие в нечетных позициях.
11	В одномерном массиве, состоящем из n целочисленных элементов, вычислить: 1) произведение отрицательных элементов массива; 2) сумму положительных элементов массива, расположенных до максимального элемента. Изменить порядок следования элементов в массиве на обратный.
12	В одномерном массиве, состоящем из n целочисленных элементов, вычислить: 1) номер максимального по модулю элемента массива; 2) сумму элементов массива, расположенных после первого положительного элемента. Преобразовать массив таким образом, чтобы сначала располагались все элементы, целая часть которых лежит в интервале $[a, b]$, а потом – все остальные.
13	В одномерном массиве, состоящем из n целочисленных элементов, вычислить: 1) количество элементов массива, лежащих в диапазоне от A до B; 2) сумму элементов массива, расположенных после максимального элемента. Упорядочить элементы массива по убыванию модулей элементов.
14	В одномерном массиве, состоящем из n целочисленных элементов, вычислить: 1) количество элементов массива, равных 0; 2) сумму элементов массива, расположенных после минимального элемента. Упорядочить элементы массива по возрастанию модулей элементов.

6.8. Лабораторная работа 9. Числовые двумерные массивы

Цель: иметь представление об организации в памяти двумерных массивов, уметь объявлять числовой двумерный массив, уметь его заполнять, выводить в консоль в виде таблицы, обращаться к элементам двумерного массива, уметь выполнять обработку элементов двумерного массива.

Задание. Разработать программы для решения задач в соответствии с вариантом.

Вариант 1

1. Заполнить двумерный массив числами с клавиатуры и вывести его на экран в виде таблицы.

2. Заполнить двумерный массив случайными числами и вывести его на экран в виде таблицы.



3. В массиве из предыдущей задачи отыскать максимальный элемент, вывести его на экран с комментарием и указать, в какой строке и каком столбце он расположен.

4. В массиве из предыдущей задачи найти суммы элементов по строкам.

5. В матрице A поэлементно вычесть последнюю строку из всех строк, кроме последней.

6. Заполнить квадратную матрицу случайными числами. Найти произведение элементов главной диагонали.

7. Заполнить таблицу целыми числами. Отыскать в ней максимальный и минимальный элементы. Поменять их местами. Вывести полученную таблицу на экран.

Вариант 2

1. Заполнить двумерный массив числами с клавиатуры и вывести его на экран в виде таблицы.

2. Заполнить двумерный массив случайными числами и вывести его на экран в виде таблицы.

3. В массиве из предыдущей задачи отыскать минимальный элемент, вывести его на экран с комментарием и указать, в какой строке и каком столбце он расположен.

4. В массиве из предыдущей задачи найти произведения элементов по столбцам.

5. Вычислить среднее арифметическое четных элементов прямоугольной матрицы У.

6. Заполнить квадратную матрицу случайными числами. Найти среднее арифметическое элементов главной диагонали.

7. Заполнить таблицу целыми числами. Отыскать в ней максимальный и минимальный элементы. Поменять их местами. Вывести полученную таблицу на экран.

6.9. Лабораторная работа 10. Числовые двумерные массивы

Цель: иметь представление об организации в памяти двумерных массивов, уметь объявлять числовой двумерный массив, уметь его заполнять, выводить в консоль в виде таблицы, обращаться к элементам двумерного массива, уметь выполнять обработку элементов двумерного массива.

Задание. Разработать программы для решения задач в соответствии с вариантом.



Таблица 6.3

Варианты заданий

Вариант	Условие задачи
1	1. Заполнить произвольный массив размером $N \times N$ ($N < 10$) по следующему правилу: 00001 00010 00100 01000 10000 2. Дана матрица A - найти номер минимального элемента в третьем столбце - заменить min элемент в 4-ом столбце на min элемент третьего столбца - получить среднеарифметическое значение последних 10-ти элементов матрицы - преобразовать матрицу в вектор
2	1. Заполнить произвольный массив размером $N \times N$ ($N < 10$) по следующему правилу: 10000 01000 00100 00010 00001 2. Дана матрица A - найти и переставить минимальный и максимальный элементы матрицы - определить число положительных элементов во второй и третьей строках - получить среднеарифметическое значение элементов матрицы над главной диагональю - заменить элементы главной диагонали на 0
3	1. Заполнить произвольный массив размером $N \times N$ ($N < 10$) по следующему правилу: 11111 10001 10001 10001 11111 2. Дан матрица A - найти в нечетных столбцах произведение положительных элементов - переставить местами минимальный и максимальный элементы матрицы - получить среднеарифметическое значение отрицательных элементов матрицы - в четвертой строке найти сумму четных чисел
4	1. Заполнить произвольный массив размером $N \times N$ ($N < 10$) по следующему правилу: 10001 00000 00000 00000 10001



	2. Дан матрица А - найти количество положительных элементов на обратной диагонали - найти сумму отрицательных элементов на главной диагонали - заменить минимальный элемент матрицы суммой элементов матрицы - определить номера минимальных элементов в третьем столбце и второй строке
5	1. Заполнить произвольный массив размером N x N ($N < 10$) по следующему правилу: 10000 02000 00300 00020 00001 2. Дана матрица - переставить элементы главной и обратной диагонали - найти сумму элементов второй и третьей строки - превратить матрицу в вектор - получить среднеарифметическое значение элементов 3-го и 5-го столбцов
6	1. Заполнить произвольный массив размером N x N ($N < 10$) по следующему правилу: 12345 54321 12345 54321 12345 2. Дана матрица А - найти сумму элементов четных строк, лежащих в диапазоне от -6 до 11. - найти min элемент и max элемент матрицы. - поменять местами min и max элементы матрицы - удалить столбец, где находится max элемент матрицы
7	1. Заполнить произвольный массив размером N x N ($N < 10$) по следующему правилу: 12345 01234 00123 00012 00001 2. Дана матрица А - найти сумму элементов в четных строках - найти максимальный элемент - напечатать сумму столбца, где находится максимальный элемент - переставить 3-ий и 5-ый столбцы
8	1. Заполнить произвольный массив размером N x N ($N < 10$) по следующему правилу: 10000 00001 10000 00001 10000



	2. Дана матрица A - найти сумму четных элементов, лежащих в диапазоне от -6 до 11 - найти min элемент 3-ей строки и max элемент главной диагонали - найти среднеарифметическое положительных элементов матрицы - поменять местами min элемент 3-ей строки и max элемент главной диагонали матрицы
9	1. Заполнить произвольный массив размером N x N ($N < 10$) по следующему правилу: <pre> 11111 22222 33333 44444 55555 </pre> 2. Дана матрица - получить сумму и количество нечетных элементов. - найти min и max элементы матрицы. - получить среднее арифметическое значение отрицательных элементов матрицы - переставить строки с min и max элементами матрицы
10	1. Заполнить произвольный массив размером N x N ($N < 10$) по следующему правилу: <pre> 12345 12345 12345 12345 12345 </pre> 2. Дана матрица A - подсчитать количество четных элементов в нечетных строках - переставить прямую и обратную диагонали матрицы - получить сумму элементов матрицы - переставить max и min элементы матрицы
11	1. Заполнить произвольный массив размером N x N ($N < 10$) по следующему правилу: <pre> 12345 01234 00123 00012 00001 </pre> 2. Дан матрица A - найти в нечетных столбцах произведение положительных элементов - переставить местами минимальный и максимальный элементы матрицы - получить вектор сумм отрицательных элементов матрицы по столбцам - в четвертой строке заменить минимальный элемент на 0
12	1. Заполнить произвольный массив размером N x N ($N < 10$) по следующему правилу: <pre> 1 2 3 4 5 16 17 18 19 6 15 24 25 20 7 14 23 22 21 8 13 12 11 10 9 </pre>



	2. Дана матрица A - найти сумму элементов, попадающих в интервалы от 0 до 5 и от 5 до 15 - найти минимальный элемент в третьей строке - преобразовать матрицу в вектор - получить сумму последних 10-ти элементов матрицы
13	1. Заполнить произвольный массив размером N x N ($N < 10$) по следующему правилу: <pre> 1 2 3 4 5 10 9 8 7 6 11 12 13 14 15 ... </pre> 2. Дан матрица A - найти количество отрицательных элементов на обратной диагонали - получить вектор, содержащий количество положительных элементов в каждом столбце - найти сумму элементов до максимального элемента матрицы - определить количество отрицательных элементов в третьей строке
14	1. Заполнить произвольный массив размером N x N ($N < 10$) по следующему правилу: <pre> 11111 22220 33300 44000 50000 </pre> 2. Дан матрица A - найти в нечетных столбцах произведение положительных элементов - переставить местами минимальный и максимальный элементы матрицы - получить вектор сумм отрицательных элементов матрицы по столбцам - в четвертой строке заменить минимальный элемент на 0

6.10. Задачи для самостоятельного решения

Числовые одномерные массивы

1. В целочисленном массиве есть нулевые элементы. Создать массив из номеров этих элементов.

2. Дан массив из N целых чисел. Вычислить сумму элементов, порядковые номера которых совпадают с значениями этих элементов.

3. Дан целочисленный массив с количеством элементов N. Вывести на печать те его элементы, индексы которых являются степенями двойки: 1, 2, 4, 8, 16, ...

4. Дан массив из N вещественных чисел. Вычислить сумму чисел, порядковые номера которых являются простыми числами.

5. В заданном одномерном массиве поменять местами соседние элементы, стоящие на четных местах, с элементами, стоящими на нечетных местах.



6. Дан массив из N натуральных чисел. Создать новый массив, элементами которого будут элементы исходного массива, оканчивающиеся на цифру k .

7. Дан массив из N целых чисел. Найти сумму его элементов, расположенных между максимальным и минимальным значениями.

8. Сформировать массив простых чисел, которые не больше заданного натурального числа N .

9. Каждый солнечный день улитка, сидящая на дереве, поднимается вверх на 2 см, а каждый пасмурный день – опускается вниз на 1 см. В начале наблюдения улитка находилась в A см от земли на B -метровом дереве. Массив данных из 30 элементов содержит сведения о том, был ли соответствующий день пасмурным или солнечным. Определить местоположение улитки к концу 30-го дня наблюдения.

10. Дан линейный массив, содержащий несколько нулевых элементов. Сжать его, выбросив эти элементы.

11. Дан массив действительных чисел. Определить, сколько раз изменяется знак в этой последовательности и отметить номера позиций, в которых происходит смена знака.

12. Заменить все элементы вектора $x(m)$, стоящие до максимального, нулями.

13. В векторе $x(n)$ поменять местами максимальный и последний элементы.

14. Найти среднее арифметическое элементов массива $x(n)$, исключив из них максимальное и минимальное значение.

15. Даны два массива одного размера. Получить третий массив, каждый элемент которого равен:

16. а) сумме элементов с тем же номером в заданных массивах;

17. б) произведению элементов с тем же номером в заданных массивах;

18. в) максимальному из элементов с тем же номером в заданных массивах.

19. Дан массив целых чисел. Переписать его нечетные элементы в другой массив такого же размера, расположив элементы подряд с начала массива.

20. Дан массив из 20 элементов. Сформировать два массива размером 10, включив в первый из них элементы заданного массива с четными индексами, а во второй – с нечетными

21. Расположить элементы вектора в обратном порядке. Дополнительный массив не использовать.

22. Определить индексы четных элементов массива $A(k)$.

23. Заполнить линейный массив числами 0, 1 и 2 в случайном порядке. Изменить расположение элементов в массиве так чтобы сначала шли 0, затем 1 и в последнюю очередь 2.

24. В массиве $R(k)$ найти минимальный элемент среди положительных и максимальный среди отрицательных.



25. Размеры 15-типараллелепипедов (длина, ширина, высота) хранятся в трех массивах. Определить: а) максимальный объем фигуры; в) номер фигуры, имеющей максимальный объем.

26. Даны два вектора А и В. Сформировать вектор С, который содержит элементы, присутствующие в обоих массивах.

27. Дано натуральное число n ($n \leq 999999$). Заполнить массив его цифрами, расположенными в обратном порядке (первый элемент равен последней цифре, второй – предпоследней и т.д.). Незаполненные элементы массива должны быть равны нулю. Элементы массива, являющиеся цифрами числа n , вывести на экран.

Числовые двумерные массивы

1. Квадратный массив $N \times N$ заполнить следующим образом: элементы главной диагонали (слева направо сверху вниз) заполнены единицами. Ниже главной диагонали – нули, а элементы выше равны сумме своих индексов.

2. В заданной целочисленной матрице $A(N, M)$ найти количество строк, содержащих нули.

3. В заданной матрице $A(N, M)$ поменять местами строки с номерами P и Q ($1 \leq P \leq N$, $1 \leq Q \leq N$).

4. По трём заданным матрицам $A(N, N)$, $B(N, N)$ и $C(N, N)$ постройте матрицу X того же размера, каждый элемент которой представляет собой среднее арифметическое соответствующих элементов исходных матриц.

5. Дана матрица $A(N, N)$. Если хотя бы один элемент строки матрицы отрицателен, то все элементы этой строки заменить нулями.

6. Заполнить квадратный массив случайными числами. Организовать два вектора. В один записать элементы матрицы, лежащие выше главной диагонали, в другой – ниже главной диагонали.

7. Дан массив $A[1..N, 1..N]$. Составить программу, которая прибавила бы каждому элементу данной строки элемент, принадлежащий этой строке и главной диагонали.

8. Создать вектор M , содержащий количество отрицательных элементов каждого столбца матрицы $z(3, 4)$.

9. Заполнить двумерный массив $T[1..n, 1..n]$ последовательными целыми числами от 1, расположенными по спирали, начиная с левого верхнего угла и продвигаясь по часовой стрелке

10. В массиве $A[1..N, 1..N]$ определить номера строки и столбца какой-нибудь седловой точки. Некоторый элемент массива называется седловой точкой, если он является одновременно наименьшим в своей строке и наибольшим в своем столбце.



Глава 7. СИМВОЛЫ И СТРОКИ

Обработка текстовой информации является одной из самых распространенных задач современного программирования. C# предоставляет для ее решения широкий набор средств: символы *char*, неизменяемые строки *string*, изменяемые строки *StringBuilder*.

7.1. Символы *char*

Тип *char* предназначен для хранения символа в кодировке *Unicode*. Символьный тип относится к встроенным типам данных C# и соответствует стандартному классу *Char* библиотеки *.NET* из пространства имен *System*. В этом классе определены статические методы, которые приведены в таблице 7.1.

Таблица 7.1

Основные методы класса *Char*

Метод	Описание
GetNumericValue	Возвращает числовое значение символа, если он является цифрой, и 1 – в противном случае
GetUnicodeCategory	Возвращает категорию Unicode-символа: цифры – <i>Decimal-DigitNumber</i> , римские цифры – <i>LetterNumber</i> , буквы в нижнем регистре – <i>LowercaseLetter</i> и др.
IsControl	Возвращает true, если символ является управляющим («\n», «\t», «\r»)
IsDigit	Возвращает true, если символ является десятичной цифрой
IsLetter	Возвращает true, если символ является буквой
IsLetterOrDigit	Возвращает true, если символ является буквой или десятичной цифрой
IsLower	Возвращает true, если символ задан в нижнем регистре
IsNumber	Возвращает true, если символ является числом (десятичным или шестнадцатеричным)
IsPunctuation	Возвращает true, если символ является знаком препинания
IsSeparator	Возвращает true, если символ является разделителем
IsUpper	Возвращает true, если символ задан в верхнем регистре
IsWhiteSpace	Возвращает true, если символ является пробельным (пробел, перевод строки, возврат каретки)
Parse	Преобразует строку в символ, строка должна состоять из одного символа
ToLower	Преобразует символ в нижний регистр
ToUpper	Преобразует символ в верхний регистр



Примеры использования методов представлены на рисунке 7.1.

```
char simvol = char.Parse(Console.ReadLine()); //ввод символа
Console.WriteLine(simvol); //вывод символа

//вывод цифры или -1 если это буква
Console.WriteLine(char.GetNumericValue(simvol));

//вывод категории символа
Console.WriteLine(char.GetUnicodeCategory(simvol));

if (char.IsLetter(simvol)) Console.WriteLine("это буква");
if (char.IsPunctuation(simvol)) Console.WriteLine("это знак препинания");

//вывод символа, преобразованного к верхнему регистру
Console.WriteLine(char.ToUpper(simvol));
```

Рис. 7.1. Код использования методов класса Char

Используя символьный тип можно организовать массив символов и работать с ним на основе базового класса *Array* (рис. 7.2).

```
//формирование массива из символов
char[] C = { 'm', 'a', 'x', 'i', 'm', 'u', 's' };
//вывод массива
foreach (var item in C)
{
    Console.Write(item);
}
//преобразование всех символов из массива в верхний регистр
for (int i = 0; i < C.Length; i++)
{
    C[i] = Char.ToUpper(C[i]);
}
//формирование массива из символов строки
char[] B = "Мама мыла раму".ToCharArray();
Console.WriteLine(B[2]); //вывод символа из массива с индексом 2
Array.Reverse(B); //переворачивание массива
```

Рис. 7.2. Код использования массивов символов

7.2. Строковый *mun string*

Тип *string* предназначен для работы со строками символов в кодировке *Unicode*, является встроенным типом в C#. Ему соответствует базовый тип класса *System.String* библиотеки .NET. Тип *string* относится к ссылочным типам.



Каждый объект данного класса – неизменяемая последовательность символов *Unicode*. Любое действие со строкой ведет к тому, что создается копия строки, в которой и выполняются все изменения. Исходная же строка не меняется. Такой подход обусловлен необходимостью сделать работу со строками максимально быстрой и безопасной.

Создать объект типа *string* можно несколькими способами, приведенными в таблице 7.2.

Таблица 7.2

Способы создания объекта типа *string*

Код	Описание
<code>string s;</code>	Инициализация отложена
<code>string s = "Мама мыла раму";</code>	Инициализация строковым литералом
<code>string s = @"Мама мыла раму";</code>	Символ @ сообщает, что строку нужно воспринимать буквально, даже если она занимает несколько строк
<code>int x = 12344556; string s = x.ToString();</code>	Инициализация целочисленной переменной и преобразование ее к типу <i>string</i>
<code>string s = new string(' ', n)</code>	Конструктор создает строку из n пробелов
<code>char [] A = {'a','b','c','d'}; string V = new string(A);</code>	Создание массива символов, затем создание строки из массива символов
<code>char [] A = {'a','b','c','d'}; string V = new string(A, k, n);</code>	Создание строки из части массива символов, начиная с k-го символа, длиной n символов

С объектом типа *string* можно работать посимвольно как с элементами одномерного массива.

В примере приведен код, в котором обрабатываемая строка вводится с клавиатуры, а затем выполняется число вхождений в нее буквы 'a' (рис. 7.3).

```
string S = Console.ReadLine();
int count = 0;
for (int i = 0; i < S.Length; i++)
{
    if (S[i]=='a')
    {
        count++;
    }
}
Console.WriteLine($"Число вхождений буквы 'a' = {count} ");
```

Рис. 7.3. Подсчет числа вхождений символа в строку

Класс *string* обладает богатым набором методов для сравнения строк, поиска в строке и др. Данные методы приведены в таблице 7.3.



Методы класса `string`

Название	Вид	Описание
<code>Compare</code>	Статический	Сравнение двух строк в лексикографическом порядке
<code>CompareTo</code>	Экземплярный	Сравнение текущего экземпляра строки с другой строкой
<code>Concat</code>	Статический	Слияние произвольного числа строк
<code>Copy</code>	Статический	Создание копии строки
<code>Empty</code>	Статический	Открытое статическое поле, представляющее пустую строку
<code>Format</code>	Статический	Форматирование строки в соответствии с заданным форматом
<code>IndexOf</code> <code>LastIndexOf</code>	Экземплярный	Определение индекса первого/последнего вхождения подстроки в данную строку
<code>IndexOfAny</code> <code>LastIndexOfAny</code>	Экземплярный	Определение индекса первого/последнего вхождения любого символа из подстроки в данную строку
<code>Insert</code>	Экземплярный	Вставка подстроки в заданную позицию
<code>Join</code>	Статический	Слияние массива строк в единую строку, между элементами вставляются разделители
<code>Length</code>	Свойство	Возвращает длину строки
<code>PadLeft</code> <code>PadRight</code>	Экземплярный	Выравнивают строки по левому/правому краю путем вставки пробелов
<code>Remove</code>	Экземплярный	Удаление подстроки из заданной позиции
<code>Replace</code>	Экземплярный	Замена всех вхождений подстроки или символа новыми подстрокой или символом
<code>Split</code>	Экземплярный	Разделяет строку на элементы, используя разные разделители, результаты помещаются в массив строк
<code>StartWith</code> <code>EndWith</code>	Экземплярный	Возвращают <code>true</code> или <code>false</code> в зависимости от того, начинается/заканчивается строка заданной подстрокой
<code>Substring</code>	Экземплярный	Выделение подстроки, начиная с заданной позиции
<code>ToCharArray</code>	Экземплярный	Преобразует строку в массив символов
<code>ToLower</code> <code>ToUpper</code>	Экземплярный	Преобразует строку к нижнему/верхнему регистру
<code>Trim</code> , <code>TrimStart</code> <code>TrimEnd</code>	Экземплярный	Удаляет пробелы в начале и конце строки/в начале/в конце

Вызов статических методов происходит через обращение к имени класса (`String.Concat(s1, s2)`), а вызов экземплярных методов через обращение к имени объекта (`str.ToLower()`). Практически каждый метод имеет перегруженную версию. Поэтому важно ознакомиться с



подсказками VS при обращении к методу. Все методы возвращают ссылку на новую строку.

Рассмотрим несколько примеров использования методов класса *string*. При преобразовании исходной строки необходимо вводить дополнительные строковые переменные для сохранения результата (рис. 7.4).

```
//определение длины строки
int n = S.Length;
//удаление из строки подстроки
string R = S.Remove(0, 4);
//создание прописной версии строки
string SUp = S.ToUpper();
//замена подстроки пустой строкой
string Zam = S.Replace("мама", "");
//конкатенация
string RS = String.Concat(S, R);
```

Рис. 7.4. Преобразование исходной строки

Результатом поиска подстроки в строке является индекс символа, с которого подстрока в строке начинается (рис. 7.5).

```
//поиск подстроки в строке
int x;
x = S.IndexOf("мама");
Console.WriteLine($"индекс первого вхождения {x}");
x = S.LastIndexOf("мама");
Console.WriteLine($"индекс последнего вхождения {x}");
```

Рис. 7.5. Поиск подстроки в строке

Очень важными методами обработки строк являются методы *Split*, разделяющий строку на элементы (рис. 7.6), и *Join*, сливающий массив строк в единую строку (рис. 7.7).

```
string poem = "тучки небесные-вечные странники";
char[] div = { ' ', '-' }; //массив разделителей
string[] word = poem.Split(div); //разделение строки
//вывод массива из слов строки
for (int i = 0; i < word.Length; i++)
{
    Console.WriteLine(word[i]);
}
```

Рис. 7.6. Разбиение строки на массив слов

```
//собираем массив слов в одну строку
string Rez = String.Join("|", word);
Console.WriteLine(Rez);
```

Рис. 7.7. Формирование строки из массива слов



7.3. Строковый *mun StringBuilder*

Определен в пространстве имен *System.Text*. Предназначен для создания строк, значения которых можно изменять. Объекты данного класса всегда создаются с помощью явного вызова конструктора класса (операции *new*). Создать объект класса *StringBuilder* возможно несколькими способами, которые приведены в таблице 7.4.

Таблица 7.4

Способы создания объекта класса *StringBuilder*

Код	Описание
<code>StringBuilder a = new StringBuilder()</code>	Создание пустой строки, рассчитанной на 16 символов
<code>StringBuilder a = new StringBuilder("abcd")</code>	Создание строки "abcd", буфер которой рассчитан на 16 символов
<code>StringBuilder a = new StringBuilder(100)</code>	Создание пустой строки, буфер которой рассчитан на 100 символов
<code>StringBuilder a = new StringBuilder("abcd", 100)</code>	Создание строки "abcd", буфер которой рассчитан на 100 символов
<code>StringBuilder a = new StringBuilder("abcdef", 1, 3, 100)</code>	Создание строки "bcd", буфер которой рассчитан на 100 символов

С объектами класса *StringBuilder*, так же как с одномерными массивами и объектами типа *String* можно работать посимвольно. Класс *StringBuilder* реализует разнообразные свойства и методы, приведенные в таблице 7.5.

Таблица 7.5

Методы и свойства класса *StringBuilder*

Название	Вид	Описание
Append	экземплярный	Добавление данных в конец строки (массивы символов, строки, подстроки)
AppendFormat	экземплярный	Добавление форматированной строки в конец данной строки
Capacity	свойство	Получить и установить емкость буфера
Insert	экземплярный	Вставка подстроки в заданную позицию
Length	свойство	Возвращает длину строки
MaxCapacity	свойство	Возвращает наибольшее количество символов, которое может быть размещено в строке
Remove	экземплярный	Удаление подстроки из заданной позиции
Replace	экземплярный	Замена всех вхождений заданной подстроки или символа новой подстрокой или символом
ToString	экземплярный	Преобразует в строку типа <i>string</i>
Chars	свойство	Обращение к элементу строки по его номеру
Equals	экземплярный	Сравнивает значения двух строк
CopyTo	экземплярный	Копирует подмножество символов строки в массив <i>char</i>



Примеры использования отдельных методов класса *StringBuilder* представлены на рисунке 7.8.

```
//создание объекта класса
StringBuilder str = new StringBuilder("Площадь");
str.Append(" треугольника равна");//добавление в конец строки
str.Insert(8, "данного");//вставка в строку
str.Remove(7, 21);//удаление подстроки
str.Replace("а", "...");//замена
str.Length = 0;//очистка строки
```

Рис. 7.8. Использование методов класса *StringBuilder*

7.4. Лабораторная работа 11. Символы и строки

Цель: иметь представление о классах для работы с символами и строками в языке программирования C#, уметь выполнять обработку символов и строк с помощью различных классов.

Задание. Разработать программы для решения задач в соответствии с вариантом.

Таблица 7.6

Варианты заданий

Вариант	Работа с символами строки	Работа с символами	Посимвольная обработка строк. Методы
1	Из слова <i>яблоко</i> путем "вырезок" и "склеек" его букв получить слова <i>блок</i> и <i>око</i> .	Дан символ <i>C</i> . Вывести его код (то есть номер в кодовой таблице).	Дана строка. Подсчитать количество содержащихся в ней цифр.
2	Из слова <i>информатика</i> путем "вырезок" и "склеек" его букв получить слова <i>форма</i> и <i>тик</i> .	Дано целое число $N(32 \leq N \leq 126)$. Вывести символ с кодом, равным <i>N</i> .	Дана строка. Подсчитать количество содержащихся в ней прописных латинских букв.
3	Из слова <i>вертикаль</i> путем "вырезок" и "склеек" его букв получить слова <i>тир</i> и <i>ветка</i> . <i>Строки символов 99</i>	Дан символ <i>C</i> . Вывести два символа, первый из которых предшествует символу <i>C</i> в кодовой таблице, а второй следует за символом <i>C</i> .	Дана строка. Если она представляет собой запись целого числа, то вывести 1, если вещественного (с дробной частью) – вывести 2; если строку нельзя преобразовать в число, то вывести 0. Считать, что дробная часть вещественного числа отделяется от его целой части десятичной точкой «.».



4	Из слова <i>вертикаль</i> путем "вырезок" и "склеек" его букв получить слова <i>тир</i> и <i>ветка</i> . <i>Строки символов 99</i>	Дано целое число $N(1 \leq N \leq 26)$. Вывести N первых прописных (то есть заглавных) букв латинского алфавита.	Даны целые положительные числа $N1$ и $N2$ и строки $S1$ и $S2$. Получить из этих строк новую строку, содержащую первые $N1$ символов строки $S1$ и последние $N2$ символов строки $S2$ (в указанном порядке).
5	Дано слово, состоящее из четного числа букв. Вывести на экран его первую половину, не используя оператор цикла.	Дано целое число $N(1 \leq N \leq 26)$. Вывести N последних строчных (то есть маленьких) букв латинского алфавита в обратном порядке (начиная с буквы «z»)	Даны строки S и $S0$. Найти количество вхождений строки $S0$ в строку S .
6	Дано слово. Получить его часть, образованную идущими подряд буквами, начиная с m -й и кончая n -й.	Дан символ C , изображающий цифру или букву (латинскую или русскую). Если C изображает цифру, то вывести строку «digit», если латинскую букву – вывести строку «lat», если русскую – вывести строку «rus».	Дана строка, изображающая арифметическое выражение вида «<цифра>±<цифра>±...±<цифра>», где на месте знака операции «±» находится символ «+» или «-» (например, «4+7-2-8»). Вывести значение данного выражения (целое число).
7	Из слова <i>программа</i> путем "вырезок" и "склеек" его букв получить слова <i>ром</i> и <i>рампа</i> .	Дана непустая строка. Вывести коды ее первого и последнего символа.	Дана строка, содержащая по крайней мере один символ пробела. Вывести подстроку, расположенную между первым и вторым пробелом исходной строки. Если строка содержит только один пробел, то вывести пустую строку.
8	Из слова <i>трос</i> путем "вырезок" и "склеек" его букв получить слова <i>сорт</i> , <i>рост</i> и <i>торс</i> .	Дано целое число $N(>0)$ и символ C . Вывести строку длины N , которая состоит из символов C .	Дана строка, изображающая двоичную запись целого положительного числа. Вывести строку, изображающую десятичную запись этого же числа.
9	Из слова <i>клоун</i> путем "вырезок" и "склеек" его букв получить слова <i>уклон</i> , <i>кулон</i> и <i>колун</i> .	Дано четное число $N(>0)$ и символы $C1$ и $C2$. Вывести строку длины N , которая состоит из чередующихся символов $C1$ и $C2$, начиная с $C1$.	Даны строки S и $S0$. Удалить из строки S первую подстроку, совпадающую с $S0$. Если совпадающих подстрок нет, то вывести строку S без изменений.



10	Из слова <i>апельсин</i> путем "вырезок" и "склеек" его букв получить слово <i>спаниель</i> .	Дана строка. Вывести строку, содержащую те же символы, но расположенные в обратном порядке.	Даны строки S и $S0$. Проверить, содержится ли строка $S0$ в строке S . Если содержится, то вывести True, если не содержится, то вывести False.
11	Дано слово из четного числа букв. Поменять местами его половины	Дана непустая строка S . Вывести строку, содержащую символы строки S , между которыми вставлено по одному пробелу.	Дано целое число $N(>0)$ и строка S . Преобразовать строку S в строку длины N следующим образом: если длина строки S больше N , то отбросить первые символы, если длина строки S меньше N , то в ее начало добавить символы «.» (точка).
12	Дано слово. Перенести первые k его букв в конец.	Дана непустая строка S и целое число $N(>0)$. Вывести строку, содержащую символы строки S , между которыми вставлено по N символов «*» (звездочка).	Дана строка, изображающая десятичную запись целого положительного числа. Вывести строку, изображающую двоичную запись этого же числа.

7.5. Лабораторная работа 12. Символы и строки

Цель: иметь представление о классах для работы с символами и строками в языке программирования C#, уметь выполнять обработку символов и строк с помощью различных классов.

Задание 1. Разработать программу с использованием типа *string*, которая для заданной строки S :

Таблица 7.6

Варианты заданий

Вариант	Задание
1	Подсчитывает общее количество вхождений символов x и y .
2	Определяет, какой из двух заданных символов встречается чаще в строке
3	Определяет, имеются ли в строке два соседствующих одинаковых символа.
4	Выводит на экран символы, которые наиболее часто встречаются в строке.
5	Выводит на экран символы, которые встречаются в строке только один раз.
6	Определяет, является ли строка палиндромом.
7	Определяет, упорядочены ли по алфавиту символы строки.
8	Подсчитывает количество букв в строке.
9	Подсчитывает количество цифр в строке.
10	Подсчитывает количество знаков пунктуации в строке.
11	Подсчитывает сумму всех содержащихся цифр в строке.



12	Подсчитывает количество всех содержащихся чисел в строке.
13	Подсчитывает сумму всех содержащихся в ней чисел.
14	Находит самую длинную подстроку, состоящую только из цифр.
15	Находит самую длинную подстроку, состоящую из повторяющихся символов.

Задание 2. Разработать программу с использованием типа *StringBuilder*, которая для заданной строки *S*:

Таблица 7.7

Варианты заданий

Вариант	Задание
1	Вставляет символ <i>x</i> после каждого вхождения символа <i>y</i> .
2	Вставляет подстроку <i>x</i> после каждого вхождения подстроки <i>y</i> .
3	Удваивает каждое вхождение заданного символа <i>x</i> .
4	Удваивает каждое вхождение заданной подстроки <i>x</i> .
5	Удаляет среднюю букву, если длина строки нечетная, и две средних, если длина строки четная.
6	Удаляет все символы <i>x</i> .
7	Удаляет все цифры.
8	Удаляет все подстроки <i>substr</i> .
9	Заменяет все вхождения подстроки <i>s1</i> на подстроку <i>s2</i> .
10	Заменяет все группы стоящих рядом точек на многоточие.
11	Меняет местами первую букву со второй, третью с четвертой и т.д.
12	Меняет местами первую букву с последней, вторую с предпоследней и т.д.
13	Определяет сколько различных символов встречается в строке.
14	Удаляет все подстроки, состоящие из цифр.
15	Удаляет из строки самую длинную подстроку, состоящую из повторяющихся символов.

Задание 3. Дана строка, в которой содержится осмысленное текстовое сообщение. Слова сообщения разделяются пробелами и знаками препинания.

Таблица 7.8

Варианты заданий

Вариант	Задание
1	Вывести только те слова сообщения, в которых содержится заданная подстрока.
2	Вывести только те слова, которые содержат не более чем <i>n</i> букв.
3	Вывести только те слова, которые начинаются с прописной буквы.
4	Вывести только те слова, которые содержат хотя бы одну цифру.
5	Удалить все слова, которые заканчиваются на заданный символ.
6	Удалить все слова, которые содержат данный символ (без учета регистра).
7	Удалить все однобуквенные слова (вместе с лишними пробелами).
8	Удалить все повторяющиеся слова (без учета регистра).
9	Подсчитать сколько раз заданное слово встречается в сообщении.
10	Подсчитать, сколько слов, состоящих только из прописных букв, содержится в сообщении.
11	Найти самое длинное слово в сообщении.
12	Найти все самые длинные слова в сообщении.



13	Найти самое короткое слово в сообщении.
14	Найти все самые короткие слова в сообщении.
15	Вывести на экран все слова-палиндромы.
16	Проверить сообщение на правильность расстановки знаков препинания, если необходимо, внести изменения.
17	Вывести слова, которые встречаются только один раз.
18	Вывести слова, которые встречаются более n раз.
19	Вывести слова сообщения в алфавитном порядке.
20	Вывести слова сообщения в порядке возрастания их длин.

7.6. Задачи для самостоятельного решения

1. Программа запрашивает слово и выводит на экран коды каждой буквы данного слова.
2. Определите долю пробелов в заданной строке.
3. Введённый текст напечатать, выделив цветом все гласные.
4. Каждые n символов во введенном тексте отделите знаком `"!"`.
5. Если брать часть слова без первой буквы, без двух первых букв и так далее, получится: «Победа» – «обеда» – «беда» – «еда» – «да» – «а». Составить программу, которая проделывает это с введённым словом или предложением.
6. Вычеркните из заданного слова все буквы `"a"`.
7. Написать программу, которая запрашивает предложение и формирует массив из слов данного предложения.
8. Дан массив из слов $A(n)$. Найти все слова, начинающиеся с заданной буквы.
9. Вывести на экран первую часть таблицы кодировки, то есть символы, коды которых начинаются с нуля и заканчиваются 127.
10. В заданном тексте подсчитайте общее количество букв `"x"` и `"y"`.
11. Заменить в слове все буквы `"a"` на `"б"`.
12. Написать программу, которая в русской фразе, введенной с клавиатуры, преобразует строчные буквы в прописные.
13. Если брать часть слова без первой буквы, без двух первых букв и так далее, получится: «Победа» – «обеда» – «беда» – «еда» – «да» – «а». Составить программу, которая проделывает это с введённым словом или предложением.
14. Вычеркните из заданного слова все буквы `"a"`.
15. Написать программу, которая запрашивает предложение и формирует массив из слов данного предложения.
16. Дан массив из слов $T(k)$. Найти все слова, содержащие заданное количество букв.



Глава 8. МЕТОДЫ

8.1. Основные понятия

Метод – это функциональный элемент класса, который реализует вычисления или другие действия, выполняемые классом или его экземпляром (объектом). Метод представляет собой законченный фрагмент кода, к которому можно обратиться по имени. Он описывается один раз, а вызываться может многократно. Совокупность методов класса определяет, что конкретно может делать класс. Например, стандартный класс `Math` содержит методы, которые позволяют вычислять значения математических функций.

Синтаксис объявления метода:

```
[атрибуты] [спецификаторы] тип_результата имя_метода ( [список формальных параметров] )  
{  
    тело_метода;  
    return значение;  
}
```

где:

1) атрибуты и спецификаторы – необязательные элементы, на данном этапе атрибуты использоваться не будут, а из всех спецификаторов обязательно будем использовать спецификатор `static`, он позволяет обращаться к методу класса без создания его экземпляра;

2) `тип_результата` определяет тип значения, возвращаемого методом, это может быть любой тип, включая типы классов, а также тип `void`, который говорит, что метод ничего не возвращает;

3) `имя_метода` будет использоваться для обращения к нему из других мест программы, и должно быть корректно заданным;

4) `список_формальных_параметров` – перечень имен переменных и их типов через запятую;

5) формальные параметры – это переменные, которые получают значения при вызове метода, если метод не имеет параметров, то список остается пустым;

6) `return` – оператор безусловного перехода, который завершает работу метода и возвращает указанное значение в точку его вызова, тип значения должен соответствовать типу результата, если `тип_результата` `void` (метод не возвращает значения), то оператор `return` либо отсутствует, либо пишется без значения.



На этапе вызова метода ему передаются фактические параметры (при наличии формальных параметров), по количеству и типу фактические параметры должны совпадать с формальными.

Таким образом, параметры используются для обмена информацией между вызывающим и вызываемым методами. В C# предусмотрено четыре типа параметров: параметры-значения, параметры-ссылки, выходные параметры и параметры, позволяющие создавать методы с переменным количеством аргументов.

При передаче параметра по значению метод получает копии параметров и операторы метода работают с этими копиями. Метод не может изменить исходные значения этих параметров.

При передаче параметров по ссылке метод получает копии адресов параметров, то есть имеет доступ к ячейкам памяти по этим адресам и может изменять их исходные значения. Чтобы параметр передавался по ссылке, нужно перед формальным параметром при описании метода и перед фактическим параметром при вызове метода ставить спецификатор `ref`.

8.2. Примеры решения задач

Задача 1. Создать метод (Func), позволяющий ввести с клавиатуры значение аргумента x , вычислить значение функции ($y=x*x$) и вывести его на экран. Осуществить неоднократный вызов метода в программе для вычисления значения функции при разных значениях аргумента.

Программный код реализации и вызова метода представлен на рисунке 8.1.

```
class Program
{
    static void Func() //дополнительный метод
    {
        Console.Write("X = ");
        double x = double.Parse(Console.ReadLine());
        double Y = x * x;
        Console.WriteLine("Y ({0}) = {1}", x, Y);
    }

    static void Main(string[] args) //точка входа в программу
    {
        Func();
        Func();
        Func();
    }
}
```

Рис. 8.1. Метод вычисления значения функции (способ 1)



Здесь метод при вызове не получает из программы входных данных и не возвращает в вызывающую программу никакого результата. Поэтому у него список формальных параметров пуст, тип результата `void` и отсутствует оператор `return`. Ввод исходных данных и вывод результата происходит внутри самого метода.

Задача 2. Создать метод (`Func`), получающий при вызове из программы значение аргумента `x` и возвращающий в основную программу значение функции ($x \cdot x$). В основной программе (в методе `Main`) организовать вызов данного метода для вычисления значения функции.

Программный код реализации и вызова метода представлен на рисунке 8.2.

```
class Program
{
    static double Func(double x) //дополнительный метод
    {
        return x * x;
    }

    static void Main(string[] args) //точка входа в программу
    {
        Console.Write("x = ");
        double x = double.Parse(Console.ReadLine());
        double y = Func(x);
        Console.WriteLine("y ({0}) = {1}", x, y);
    }
}
```

Рис. 8.2. Метод вычисления значения функции (способ 2)

Здесь ввод аргумента `x` и вывод результата на экран реализованы в основной программе. При вызове методу передается значение `x` (соответственно присутствует формальный параметр). Полученное значение функции метод возвращает в точку своего вызова (соответственно присутствует `return`).

Задача 3. Создать метод (`Func`), получающий при вызове из программы значение аргумента `x` и возвращающий в основную программу значение функции ($x \cdot x$). В основной программе (в методе `Main`) организовать вызов данного метода для вычисления значения функции для аргумента `x`, изменяющегося на интервале от `x1` до `x2` с шагом `h`.

Программный код реализации и вызова метода представлен на рисунке 8.3.

Пример аналогичен предыдущему, ввод аргумента `x` и вывод результата на экран реализованы в основной программе. При вызове методу передается значение `x` (соответственно присутствует формальный



параметр). Полученное значение функции метод возвращает в точку своего вызова (соответственно присутствует return). Только вызов метода Func организован в основной программе внутри цикла

```
class Program
{
    static double Func(double x) //дополнительный метод
    {
        return x * x;
    }

    static void Main(string[] args) //точка входа в программу
    {
        Console.WriteLine("x1 = ");
        double x1 = double.Parse(Console.ReadLine());
        Console.WriteLine("x2 = ");
        double x2 = double.Parse(Console.ReadLine());
        Console.WriteLine("h = ");
        double h = double.Parse(Console.ReadLine());

        for (double x = x1; x <= x2; x+=h)
        {
            double y = Func(x);
            Console.WriteLine("y ({0}) = {1}", x, y);
        }
    }
}
```

Рис. 8.3. Метод вычисления значения функции (способ 3)

Задача 4. Создать метод (BID), выбирающий большее из двух целых чисел. Используя метод, найти большее из трех целых чисел.

Программный код реализации и вызова метода представлен на рисунке 8.4.

```
class Program
{
    static int BID(int x, int y)
    {
        if (x>y) return x;
        else return y;
    }

    static void Main(string[] args)
    {
        Random rnd = new Random();
        int a = rnd.Next(0, 100);
        int b = rnd.Next(0, 100);
        int c = rnd.Next(0, 100);
        Console.WriteLine("{0} {1} {2} ", a, b, c);
        int max = BID(a, b);
        max = BID(max, c);
        Console.WriteLine("Большее из них = {0}", max);
    }
}
```

Рис. 8.4. Метод нахождения большего из двух чисел



Для поиска большего из трех чисел в методе Main дважды вызывается метод нахождения большего из двух чисел (BID). Сначала метод вызывается для первых двух чисел a и b, затем – для найденного max и третьего числа c (рис. 8.5).

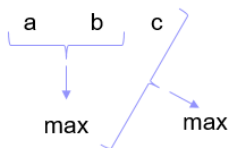


Рис. 8.5. Схема поиска большего из трех чисел

Задача 5. Создать метод (InStep), позволяющий возвести целое число в целую неотрицательную степень. С помощью метода перевести килобайт, мегабайт, гигабайт в байты.

Формулы перевода и возведения в целую неотрицательную степень приведены на рисунке 8.6.

$$\begin{aligned} 1 \text{ Кбайт} &= 2^{10} \text{ байт} \\ 1 \text{ Мбайт} &= 2^{20} \text{ байт} \\ 1 \text{ Гбайт} &= 2^{30} \text{ байт} \end{aligned}$$

$$a^n = \underbrace{a * a * \dots * a}_{n \text{ раз}}$$

Рис. 8.6. Используемые формулы

Программный код реализации и вызова метода представлен на рисунке 8.7.

```
class Program
{
    static int InStep (int a, int n)
    {
        int Rez = 1;
        for (int i = 1; i <= n; i++)
        {
            Rez *= a;
        }
        return Rez;
    }

    static void Main(string[] args)
    {
        int KB = InStep(2, 10);
        int MB = InStep(2, 20);
        int GB = InStep(2, 30);
        Console.WriteLine("1 Кбайт = {0} байт", KB);
        Console.WriteLine("1 Мбайт = {0} байт", MB);
        Console.WriteLine("1 Гбайт = {0} байт", GB);
    }
}
```

Рис. 8.7. Метод возведения числа в целую неотрицательную степень



8.3. Перегрузка методов

Представляется разумным, чтобы методы, реализующие один и тот же алгоритм для различного количества параметров или различных типов данных, имели одно и то же имя. Использование таких методов называется перегрузкой методов. Компилятор определяет, какой именно метод вызвать, по типу и количеству фактических параметров.

Перегрузка методов является проявлением полиморфизма, одного из основных принципов объектно-ориентированного программирования.

8.4. Лабораторная работа 13. Методы

Цель: иметь представление о методах, знать правила создания и вызова методов, уметь передавать значения в вызываемые методы, уметь разрабатывать программы с использованием вспомогательных методов.

Задание. Разработать программы с использованием вспомогательных методов, выполняющих часть основной задачи, в соответствии с вариантом.

Вариант 1

1. Написать метод вычисления факториала натурального числа n (формальным параметром метода является натуральное число n , метод возвращает значение факториала). Факториалом числа n называется число, равное произведению целых чисел от 1 до n . Организовать вызов данного метода в основной программе (методе Main) для значения n , вводимого с клавиатуры.

2. Четырехугольник задан длинами своих сторон и диагонали. В основной программе вычислить площадь четырехугольника (как сумму площадей двух треугольников, его образующих) с использованием дополнительного метода вычисления площади треугольника по трем сторонам (формула Герона). Формальными параметрами метода являются длины сторон треугольника, метод возвращает в точку своего вызова значение площади треугольника.

3. Написать программу для вычисления значения выражения $f(0.8,a)+f(a,b)-f(2a-1,ab)$, где a, b действительные числа. Использовать вспомогательный метод вычисления значения функции вида

$$f(x,y) = \frac{x^2 + y^2}{x^2 + 2xy + y^2 + 6}$$

Вариант 2

1. В основной программе (методе Main) найти минимальное среди 4-х целых чисел, используя вспомогательный метод нахождения минимального среди двух целых чисел.



2. Треугольник задан координатами своих вершин. В основной программе вычислить площадь треугольника по формуле Герона с использованием дополнительного метода нахождения расстояния между двумя точками на координатной плоскости. Формальными параметрами метода являются координаты двух точек на плоскости – концов отрезка, метод возвращает в точку своего вызова длину отрезка на плоскости.

3. Вычислить значение выражение $\lg^2 x + 2\lg x$, используя вспомогательный метод вычисления тангенса угла.

8.5. Лабораторная работа 14. Методы

Цель: иметь представление о методах, знать правила создания и вызова методов, уметь передавать значения в вызываемые методы, уметь разрабатывать программы с использованием вспомогательных методов.

Задание. Разработать программы с использованием вспомогательных методов, выполняющих часть основной задачи, в соответствии с вариантом. Для задачи 2 разработать отдельный метод для формирования массива, вывода в консоль, а также для каждого пункта.

Таблица 8.1

Варианты заданий

Вариант	Задание
1	1. Четырехугольник задан длинами своих сторон a, b, c, d и диагонали g. Вычислить площадь четырехугольника (как сумму площадей двух треугольников, его образующих) с использованием подпрограммы вычисления площади треугольника по трем сторонам (формула Герона). 2. Дан вектор $A[1:20]$ - найти сумму положительных чисел и сумму отрицательных чисел - найти произведение этой суммы на $\min$ элемент вектора - найти сумму элементов вектора между $\min$ и последним элементом вектора
2	1. Написать программу для нахождения суммы большего и меньшего из трех чисел. Используя подпрограммы нахождения большего и меньшего из двух чисел. 2. Дан вектор $A[1:25]$ - найти минимальный элемент и его координату среди четных ячеек - найти сумму элементов вектора, больших – 2 - найти номер максимального элемента из первых 10 элементов вектора - отсортировать вектор
3	1. Создать подпрограмму, которая получает в качестве аргумента целое число и проверяет, является ли оно совершенным. Используя подпрограмму вывести на экран все совершенные числа из промежутка целых чисел, определенного пользователем, и их общее количество. Совершенное число представляет собой сумму всех своих делителей, меньших его самого. 2. Дан вектор $A[1:25]$ - отсортировать вектор по убыванию - удалить минимальный элемент вектора - получить среднеарифметическое значение элементов с четными индексами - поменять местами максимальные элементы из первых десяти и последних десяти элементов вектора



4	1. Два простых числа называются «близнецами», если они отличаются друг от друга на 2 (например, 41 и 43). Напечатать все пары «близнецов» из отрезка $[p, 2n]$, где p – заданное натуральное число больше 2. 2. Дан вектор $B[1:25]$ - переставить местами первый и максимальный элементы вектора - преобразовать вектор в матрицу - заменить максимальный элемент вектора среднеарифметическим значением последних 15 элементов - напечатать элементы вектора с последнего до максимального
5	1. Среди трехзначных чисел найти такие, в которых сумма факториалов цифр равнялась бы самому числу. Использовать подпрограмму вычисления факториала натурального числа. ($n! = 1 * 2 * 3 * \dots * n$). 2. Дан вектор $B[1:20]$ - найти минимальный и максимальный элементы вектора - поменять местами максимальный и минимальный элементы - отсортировать вектор по возрастанию - подсчитать количество отрицательных элементов вектора
6	1. На плоскости заданы своими координатами n точек. Составить программу, определяющую, между какими из пар точек самое большое расстояние. Координаты точек занести в массив. 2. Дан вектор из 20 элементов - получить сумму нечетных и положительных элементов вектора - найти минимальный элемент вектора среди четных ячеек - получить новый вектор, содержащий только четные числа исходного вектора - напечатать вектор с последнего до первого
7	1. Разработать подпрограмму, которая в качестве аргумента получает целое число и проверяет, является ли оно простым. Используя подпрограмму вывести на экран все простые числа из промежутка целых чисел, определенного пользователем, и их общее количество. Простое число делится без остатка лишь на единицу и на себя. 2. Дан вектор $B[1:25]$ - найти сумму элементов в диапазоне от -2 до 2 - отсортировать вектор по убыванию - получить произведение $\min$ и $\max$ элементов вектора - получить сумму элементов между $\max$ и последним элементами вектора
8	1. Два натуральных числа называются «дружественными», если каждое из них равно сумме всех делителей (кроме его самого) другого числа (например, числа 220 и 284). Найти все пары «дружественных чисел», которые не больше данного числа N. 2. Дан вектор $B[1:25]$ - найти сумму четных элементов в четных ячейках - переставить местами $\max$ и первый элемент вектора - отсортировать вектор по убыванию - получить сумму элементов между последним и $\max$ элементами вектора
9	1. Написать программу для вычисления суммы факториалов всех нечетных чисел от 1 до 9. Для решения использовать подпрограмму вычисления факториала натурального числа. ($n! = 1 * 2 * 3 * \dots * n$). 2. Дан вектор из 15 элементов - получить сумму элементов кратных 3 - отсортировать вектор - определить номер минимального элемента из первых 5 элементов и номер $\max$ из последних 5 элементов



10	1. Вычислить площадь правильного шестиугольника со стороной, a, используя подпрограмму вычисления площади правильного треугольника. 2. Дан вектор $A[1:20]$ - найти количество четных элементов - найти номер $\min$ элемента с 1 по 10 и номер $\max$ элемента с 11 по 20 элемент вектора - удалить все нечетные числа вектора - получить среднеарифметическое значение последних 10-ти элементов вектора
11	1. Даны n натуральных чисел. Найти их наибольший общий делитель, используя алгоритм Евклида. Создать метод, определения НОД двух натуральных чисел. 2. Дан вектор $A[1:20]$ - найти максимальный элемент из последних десяти - напечатать вектор до $\min$ элемента - удалить $\min$ элемент вектора - получить сумму элементов вектора между последним и $\max$ элементом
12	1. Вывести на экран все сверхпростые числа из промежутка целых чисел, определенного пользователем, и их общее количество. Сверхпростым считается число, если оно само является простым и его перевертыш тоже является простым. 2. Дан вектор $B[1:25]$ - найти сумму четных элементов в четных ячейках - переставить местами $\max$ и первый элемент вектора - отсортировать вектор по убыванию - получить сумму элементов между последним и $\max$ элементами вектора
13	1. Даны 6 различных чисел. Найти максимальное из них. Разработать вспомогательный метод, позволяющий определить наибольшее число из двух. 2. Дан вектор $A[1:25]$ - найти сумму последних 10 элементов - удалить минимальный элемент вектора - получить среднеарифметическое значение элементов с четными индексами - поменять местами максимальные элементы из первых десяти и последних десяти элементов вектора

8.6. Задачи для самостоятельного решения

1. Написать программу для нахождения суммы большего и меньшего из трех чисел. Используя методы нахождения большего и меньшего из двух чисел.

2. Вычислить площадь правильного шестиугольника со стороной a , используя метод вычисления площади правильного треугольника.

3. Среди трехзначных чисел найти такие, в которых сумма факториалов цифр равнялась бы самому числу. Использовать метод вычисления факториала натурального числа. ($n! = 1 * 2 * 3 * \dots * n$).

4. Написать программу для вычисления суммы факториалов всех нечетных чисел от 1 до 9. Для решения использовать метод вычисления факториала натурального числа. ($n! = 1 * 2 * 3 * \dots * n$).



5. Создать метод, который получает в качестве аргумента целое число и проверяет, является ли оно совершенным. Используя метод, вывести на экран все совершенные числа из промежутка целых чисел, определенного пользователем, и их общее количество. Совершенное число представляет собой сумму всех своих делителей, меньших его самого.

6. Разработать метод, который в качестве аргумента получает целое число и проверяет, является ли оно простым. Используя метод, вывести на экран все простые числа из промежутка целых чисел, определенного пользователем, и их общее количество. Простое число делится без остатка лишь на единицу и на себя.

7. Два простых числа называются «близнецами», если они отличаются друг от друга на 2 (например, 41 и 43). Напечатать все пары «близнецов» из отрезка $[p, 2n]$, где p – заданное натуральное число больше 2.

8. Написать программу перевода числа из двоичной системы счисления в десятичную.

9. На плоскости заданы своими координатами n точек. Составить программу, определяющую, между какими из пар точек самое большое расстояние. Координаты точек занести в массив.

10. Два натуральных числа называются «дружественными», если каждое из них равно сумме всех делителей (кроме его самого) другого числа (например, числа 220 и 284). Найти все пары «дружественных чисел», которые не больше данного числа N .



РЕКОМЕНДАТЕЛЬНЫЙ БИБЛИОГРАФИЧЕСКИЙ СПИСОК

1. Кудрина, Е. В. Основы алгоритмизации и программирования на языке C#: учебное пособие для СПО / Е. В. Кудрина, М. В. Огнева. – М.: Юрайт, 2020. – 323 с. – Текст: непосредственный.
2. Семакин, И. Г. Основы алгоритмизации и программирования: практикум / И. Г. Семакин, А. П. Шестаков. – М.: Издательский центр «Академия», 2014. – 141 с. – Текст: непосредственный.
3. Хиценко, В. П. Основы программирования: учебное пособие : Утв. редакционно-издательским советом / В. П. Хиценко. – Новосибирск: Новосибирский государственный технический университет, 2015. – 83 с. – URL: <http://biblioclub.ru/index.php?page=book&id=438365> (25.10.2021). – Текст: электронный
4. Судоплатов, С. В. Математика: математическая логика и теория алгоритмов : учеб. и практ. для СПО / С. В. Судоплатов, Е. В. Овчинникова. – 5-е изд., стер. – М. : Юрайт, 2020. – 255 с. – URL: <https://urait.ru/bcode/456883> (12.09.2021). – Текст: электронный.
5. Полное руководство по C# 10 и .NET 6 // Сайт о программировании METANIT.COM. – URL: <https://metanit.com/sharp/tutorial> (31.01.2022). – Текст: электронный.



БИБЛИОГРАФИЧЕСКИЙ СПИСОК

1. Кудрина Е. В., Огнева М. В. Основы алгоритмизации и программирования на языке C#: учебное пособие для СПО. М.: Юрайт, 2020.
2. Семакин И. Г., Шестаков А. П. Основы алгоритмизации и программирования: практикум. М.: Академия, 2014.
3. Вирт Н. Алгоритмы и структуры данных: пер. с англ. СПб.: Невский диалект, 2008.
4. Кормен Т. Х. Алгоритмы: построение и анализ : пер. с англ. М.: Вильямс, 2013.
5. Казанский А. А. Программирование на Visual C# 2013: учебное пособие для прикладного бакалавриата. М.: Юрайт, 2018.
6. Павловская Т. А. C#. Программирование на языке высокого уровня: учебник для вузов. СПб.: Питер, 2009.



Учебное издание

Табаргина Галина Викторовна

**ОСНОВЫ АЛГОРИТМИЗАЦИИ
И ПРОГРАММИРОВАНИЯ НА ЯЗЫКЕ C#**

Учебно-методический комплекс по дисциплине

Лабораторный практикум



Подписано в печать с готового оригинал-макета 03.02.2022. Формат 60 × 84 1/16.
Гарнитура Times New Roman. Печать – ризограф. Бумага офсетная.
Физ. печ. л. 6,5. Усл. печ. л. 6,04. Уч.-изд. л. 4,5.
Тираж 50 экз. Заказ № 9.

Издательство ФГБОУ ВО «Хакасский государственный университет
им. Н. Ф. Катанова»
Отпечатано в типографии ФГБОУ ВО «Хакасский государственный университет
им. Н. Ф. Катанова»
655017, г. Абакан, пр. Ленина, 90А, тел. 22-51-13; e-mail: izdat@khsu.ru

